Building data course — Week 4 — FDD at scale

sparql-playground / lessons

Generated for print and offline reading

Week 4 — FDD at scale

Five daily lessons. Each day has two scripts that compute the same thing: `dayNN_pandas.py` reads the CSV export into pandas, `dayNN_sql.py` runs SQLite queries on the same rows (`uv run python lessons/course.py setup` loads them once). Run both with `uv run python lessons/course.py run N` and compare the printouts. Each lesson's two scripts are printed after it. Keep notes under the ignored `student-work/` folder.

Contents

- Day 16 — One answer three ways: a Python loop, pandas and SQL
- Day 17 — Memory: lists, generators, DataFrames and chunks
- Day 18 — Gaps and coverage: sample-and-hold, three ways
- Day 19 — Joining weather: the as-of join
- Day 20 — Scaling showdown, and the Week 4 challenge

Day 16 — One answer three ways: a Python loop, pandas and SQL

Goal: compute the same fault count and run hours with a row-by-row Python loop, with vectorized pandas and with one SQL aggregate; see that they agree; then measure how each one grows with the data, and what moving the data costs.

Read

Week 4 asks one question over and over: *the answer is the same, so which tool should compute it?* You have written Open-FDD-style rules in SQL (Unit 1) and in pandas (Day 14). Plain Python is the third option, and the one a CS course starts with.

Today's job is deliberately small, so the three versions fit on a screen. Take the FC1 raw condition from Day 14 (**fan running and duct static more than 25 Pa below setpoint**) and compute two numbers:

- `fault_rows`: how many 5-minute rows meet the condition;
- `run_hours`: fan run hours, which is rows with `fan_status = 1` times 5 minutes.

The data is **synthetic** this week, so we can make it as big as we like. `lab.synthetic_history(n)` builds `n` rows of one AHU in SI units (a daily fan schedule, duct static tracking its setpoint with noise, one weak-fan window), and it is deterministic: both scripts get identical rows. The SQL script loads them into an in-memory SQLite table first. That load is **setup**, timed on its own; the computation is what we compare.

How each one does it:

	Plain Python (in `day16_pandas.py`)	**pandas (`day16_pandas.py`)**	**SQL (`day16_sql.py`)**
Where the rows live	Python objects pulled out of a column	DataFrame columns (NumPy arrays)	table in the database
Who runs the loop	the interpreter, once per row	compiled C inside NumPy	compiled C inside SQLite
Condition	`if fan == 1 and static < sp - tol`	`(fan == 1) & (static < sp - tol)`	`CASE WHEN ... THEN 1 ELSE 0 END`
Add them up	`fault_rows += 1`	`mask.sum()`	`SUM(...)`

Big-O. All three are O(n): they touch every row once, so ten times the rows costs about ten times the time. Big-O hides the **constant factor**, and the constant is what you actually wait for. A Python loop pays interpreter overhead on every row: fetch a value, compare, update a counter. NumPy and SQLite run the same logic in compiled loops, so their per-row cost is smaller by one to two orders of magnitude. Same curve, very different slope.

The hidden cost: moving data. The pandas and SQL numbers assume the data is already where that tool wants it: in a DataFrame, or in a table. If your data lives in SQLite and you want pandas, you first pay to copy it into a DataFrame (`read_sql_query`). If it lives in a DataFrame and you want SQL, you pay to insert it (`to_sql`). Those copies are O(n) too, and they are usually *bigger* than the computation they enable. Each script times the move it can see: the pandas script, DataFrame into SQLite; the SQL script, table out into a DataFrame.

Run

```
uv run python lessons/course.py run 16
```

That runs `day16_pandas.py`, then `day16_sql.py` (about 20 seconds in all; the million-row steps dominate). Each is sectioned with numbered comments. The pandas script has version (a), a plain Python loop, and version (b), vectorized pandas:

```
def loop_answer(d):
    fault_rows = run_rows = 0
    for fan, static, sp in zip(d["fan_status"], d["duct_static"],
         d["duct_static_sp"], strict=True):
        if fan == 1:
            run_rows += 1
            if static < sp - TOL_PA:
                fault_rows += 1
    return {"fault_rows": fault_rows, "run_hours": run_rows * STEP_H}


def pandas_answer(d):
    running = d["fan_status"] == 1
    fault = running & (d["duct_static"] < d["duct_static_sp"] - TOL_PA)
    return {"fault_rows": int(fault.sum()), "run_hours":
         float(running.sum()) * STEP_H}
```

The SQL script is version (c): the loop moves inside the database and one row comes back.

```
SELECT SUM(CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
         :tol THEN 1 ELSE 0 END) AS fault_rows,
       SUM(fan_status = 1) * :step_h AS run_hours
FROM h
```

`SUM(fan_status = 1)` works because SQLite comparisons evaluate to 0 or 1. Both scripts print the same answer, for 20,000 rows and then for every benchmark size (`fault_run` and `fault_by_size` in `RESULTS`):

```
 fault_rows  run_hours
       1015    832.667
   rows  fault_rows  run_hours
   1000          73     41.333
  10000         486    419.333
 100000        4993   4164.000
1000000       50412  41664.000
```

The pandas script also prints `loop and pandas agree: True`; the unit test checks pandas against SQL.

Timing, growing the data. `lab.best_time` runs a function a few times and keeps the fastest, which hides noise from whatever else your computer is doing. The data is built once per size, outside the timers. Timings are **printed, never compared**: they differ on every machine and every run. On the machine this was written on:

```
   rows    loop_s  pandas_s  loop/pandas        (day16_pandas.py)
   1000 0.0002465  0.000226        1.091
  10000  0.002491 0.0003914        6.365
 100000   0.03065 0.0007246         42.3
1000000    0.3069   0.00555         55.3

   rows   load_s  sql_agg_s  read_sql_s  read_sql/agg
(day16_sql.py)
   1000 0.003805  0.0001054    0.001937         18.38
  10000  0.02204   0.001037     0.01824         17.58
 100000   0.2252    0.01153      0.1913         16.58
1000000    2.212      0.123       2.517         20.47
```

Read the tables in two directions. Down a column, the time grows about 10x per row once the data is big enough to matter: that's O(n). (The smallest sizes are dominated by fixed overhead: starting a query, allocating arrays.) Across a row, the three tools differ by a constant factor. At a million rows the loop took 0.31 s, the SQL aggregate 0.12 s (about 2.5x faster than the loop) and pandas 0.0055 s (about 55x faster). Both pandas and SQLite are compiled loops, but pandas only has to compare and add whole arrays in memory, while SQLite evaluates a `CASE` expression per row, decoding each row from its B-tree pages as it goes.

What did it cost to get the data there? Now the part that surprises people. The pandas script also prints what it costs to insert the DataFrame into SQLite, next to the pandas computation:

```
   rows  to_sql_s  pandas_calc_s  to_sql/calc
1000000     2.031        0.00555          366
```

And the SQL script's `read_sql_s` column above is the opposite move (SQLite table out into a DataFrame, no computation at all): 2.5 s at a million rows, about 20 times the SQL aggregate that answers the question in place. Moving data costs two to three orders of magnitude more than the pandas arithmetic, in either direction: inserting a DataFrame builds a B-tree and an index entry per row, and reading a table out builds a Python tuple per row before converting to arrays. If the data lives in SQLite and you need *one number*, "load into pandas, then compute" (`read_sql_s` plus the pandas time) loses badly to the SQL aggregate, because the aggregate runs where the data is and only one row crosses the boundary. "pandas is faster than SQL" is only true *once the data is already a DataFrame*. If you will ask fifty questions of the same data, pay the load once and use pandas. That trade-off is Day 20's main theme.

The worst version of all is the one that looks innocent: loop in Python, but over a list you built with `fetchall()`. You pay the transfer *and* the interpreter, and hold every row in memory as well. Day 17 measures that memory.

Try it

1. The loop above reads three columns. What happens to its time if it reads all six (iterating `d.itertuples(index=False, name=None)`)? Time it at 100,000 rows and say why.
2. Add a third number to all three versions: the *fault run hours*, `fault_rows * 5 / 60`. Which version needed the fewest new lines?

Answers

1. It gets slower. Timing `zip` over three columns against `itertuples` over six, both at 100,000 rows, gave about 0.033 s against 0.060 s. The cost per row has a part that depends on the row's width: Python has to build a bigger tuple (and objects for the values you never use). Fetch only the columns you need. The same advice applies to `SELECT *` into a DataFrame, and it is why the scripts here use `usecols` when they read the CSVs.
2. SQL: one more expression, `SUM(fan_status = 1 AND duct_static < duct_static_sp - :tol) * :step_h AS fault_h`. pandas: one more line, `fault_rows * STEP_H`. The loop needs no new counter here (`fault_rows` already exists), but any number that isn't a function of the existing counters needs a new counter *and* a new `if`, and every extra number makes the loop longer.

Why it matters

Open-FDD ships every rule twice (pandas oracle, SQL pack) because they are better at different jobs, not because one is always right. Today's tables are the first honest picture of that. Days 17–20 add the other axes: memory (Day 17), gaps in the data (Day 18), joining another time series (Day 19), and the full scaling showdown (Day 20).

day16_pandas.py

```
"""Day 16, pandas side: one answer two ways, a plain Python loop and
        vectorized pandas.

    uv run python lessons/unit2-python/day16_pandas.py

The question: on synthetic history (the same rows day16_sql.py loads
        into SQLite), how many
5-minute rows have the fan running with duct static more than 25 Pa
        below setpoint
(`fault_rows`), and how many fan run hours are there (`run_hours`)?

This script holds the Python loop and the pandas version, then times
        both as the data grows, and
times what it costs to copy a DataFrame into SQLite. Compare with
        day16_sql.py.
"""

import sqlite3
import sys
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

TOL_PA = 25.0  # tolerance below setpoint, pascals
STEP_H = 5 / 60  # one row = 5 minutes, in hours
N_PARITY = 20_000  # rows for the answer both scripts compare
SIZES = lab.sizes([1_000, 10_000, 100_000, 1_000_000])

# 1. Data: deterministic synthetic history (the same rows in both
        scripts)
df = lab.synthetic_history(N_PARITY)
print(df.head(3).to_string(index=False))


# 2. Python side, version (a): a plain loop. The interpreter runs the
        body once per row.
def loop_answer(d):
    fault_rows = 0
    run_rows = 0
    for fan, static, sp in zip(d["fan_status"], d["duct_static"],
        d["duct_static_sp"], strict=True):
        if fan == 1:
            run_rows += 1
            if static < sp - TOL_PA:
                fault_rows += 1
    return {"fault_rows": fault_rows, "run_hours": run_rows * STEP_H}


# 3. Version (b): vectorized pandas. No loop in our code: each line is
        one whole-column operation.
def pandas_answer(d):
    running = d["fan_status"] == 1
```

```
    fault = running & (d["duct_static"] < d["duct_static_sp"] - TOL_PA)
    return {"fault_rows": int(fault.sum()), "run_hours":
         float(running.sum()) * STEP_H}


a, b = loop_answer(df), pandas_answer(df)
fault_run = pd.DataFrame([b])
print(f"\nloop   : {a}")
print(f"pandas : {b}")
print("loop and pandas agree:", a == b)

# 4. The same two numbers at every benchmark size (compared with the
         SQL script's)
by_size = pd.DataFrame([{"rows": n,
         **pandas_answer(lab.synthetic_history(n))} for n in SIZES])

# 5. Timing: grow the data, time only the computation (data built
         outside the timers)
timing, moving = [], []
for n in SIZES:
    d = lab.synthetic_history(n)
    reps = 3 if n <= 100_000 else 2
    t_loop = lab.best_time(lambda d=d: loop_answer(d), reps)
    t_pandas = lab.best_time(lambda d=d: pandas_answer(d), reps)
    timing.append({"rows": n, "loop_s": t_loop, "pandas_s": t_pandas})

    def insert(d=d):  # DataFrame -> SQLite: the cost of moving data
         *into* a database
        scratch = sqlite3.connect(":memory:")
        lab.to_sqlite(d, scratch, "h", index=["equipment_id", "ts"])
        scratch.close()

    moving.append(
        {
            "rows": n,
            "to_sql_s": lab.best_time(insert, 1 if n >= 1_000_000 else
         reps),
            "pandas_calc_s": t_pandas,
        }
    )

timing = pd.DataFrame(timing)
timing["loop/pandas"] = timing.loop_s / timing.pandas_s
timing["loop_us_per_row"] = timing.loop_s / timing.rows * 1e6
timing["pandas_us_per_row"] = timing.pandas_s / timing.rows * 1e6
moving = pd.DataFrame(moving)
moving["to_sql/calc"] = moving.to_sql_s / moving.pandas_calc_s

RESULTS = {"fault_run": fault_run, "fault_by_size": by_size}

print("\nfault rows and run hours (the loop and pandas give these):")
print(fault_run.round(3).to_string(index=False))
print(by_size.round(3).to_string(index=False))
print("\ntiming, seconds (times vary by machine; read the shape, not
         the digits):")
print(timing.to_string(index=False, float_format=lambda x: f"
         {x:,.4g}"))
print("\nwhat moving a DataFrame into SQLite costs, next to the pandas
         computation:")
```

```
print(moving.to_string(index=False, float_format=lambda x: f"
        {x:,.4g}"))
big = timing.iloc[-1]
print(f"\nobservation: at {int(big.rows):,} rows the loop is
        {big['loop/pandas']:.0f}x slower")
print(
    "observation: the loop is slowest at every size:",
        bool((timing.loop_s > timing.pandas_s).all())
)
print(
    "observation: copying into SQLite costs more than the calculation
        at every size:",
    bool((moving.to_sql_s > moving.pandas_calc_s).all()),
)
```

day16_sql.py

```
"""Day 16, SQL side: one answer as a single SQL aggregate on SQLite.

    uv run python lessons/unit2-python/day16_sql.py

The same question as day16_pandas.py on the same synthetic rows: how
        many 5-minute rows have the fan
running with duct static more than 25 Pa below setpoint (`fault_rows`),
        and how many fan run hours
are there (`run_hours`)? The rows are loaded into an in-memory SQLite
        table first; loading is setup
(timed on its own), the aggregate is the computation. Compare with
        day16_pandas.py.
"""

import sqlite3
import sys
import time
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

PARAMS = {
    "tol": 25.0,  # tolerance below setpoint, pascals
    "step_h": 5 / 60,  # one row = 5 minutes, in hours
}
N_PARITY = 20_000  # rows for the answer both scripts compare
SIZES = lab.sizes([1_000, 10_000, 100_000, 1_000_000])

# The loop moves inside the database; only one row comes back.
# SUM(fan_status = 1) works because a comparison is 0 or 1 in SQLite.
AGG_SQL = """
SELECT SUM(CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
        :tol THEN 1 ELSE 0 END)
        AS fault_rows,
       SUM(fan_status = 1) * :step_h AS run_hours
FROM h
"""


def make(n):
    """n rows of synthetic history in an in-memory table `h`, with an
        index. Setup, not compared."""
    mem = sqlite3.connect(":memory:")
    lab.to_sqlite(lab.synthetic_history(n), mem, "h", index=
        ["equipment_id", "ts"])
    return mem


# 1. Data: the same synthetic history, loaded into SQLite
conn = make(N_PARITY)
print(pd.read_sql_query("SELECT * FROM h LIMIT 3",
        conn).to_string(index=False))
```

```
# 2. The aggregate (version (c))
fault_run = pd.read_sql_query(AGG_SQL, conn, params=PARAMS)

# 3. The same two numbers at every benchmark size
by_size = []
for n in SIZES:
    mem = make(n)
    by_size.append({"rows": n, **pd.read_sql_query(AGG_SQL, mem,
        params=PARAMS).iloc[0].to_dict()})
    mem.close()
by_size = pd.DataFrame(by_size).astype({"rows": "int64", "fault_rows":
        "int64"})

# 4. Timing: load once per size (timed separately), then time only the
        aggregate and the copy out
timing = []
for n in SIZES:
    started = time.perf_counter()  # setup: build the table and its
        index, timed on its own
    mem = make(n)
    t_load = time.perf_counter() - started
    reps = 3 if n <= 100_000 else 2
    timing.append(
        {
            "rows": n,
            "load_s": t_load,
            "sql_agg_s": lab.best_time(
                lambda mem=mem: mem.execute(AGG_SQL,
        PARAMS).fetchone(), reps
            ),
            # the cost of moving the whole table *out* into a DataFrame
        (no computation at all)
            "read_sql_s": lab.best_time(
                lambda mem=mem: pd.read_sql_query("SELECT * FROM h",
        mem),
                1 if n >= 1_000_000 else reps,
            ),
        }
    )
    mem.close()
timing = pd.DataFrame(timing)
timing["read_sql/agg"] = timing.read_sql_s / timing.sql_agg_s
timing["agg_us_per_row"] = timing.sql_agg_s / timing.rows * 1e6

RESULTS = {"fault_run": fault_run, "fault_by_size": by_size}

print("\nfault rows and run hours (the SQL aggregate gives these):")
print(fault_run.round(3).to_string(index=False))
print(by_size.round(3).to_string(index=False))
print("\ntiming, seconds (times vary by machine; read the shape, not
        the digits):")
print(timing.to_string(index=False, float_format=lambda x: f"
        {x:,.4g}"))
print("\nthe aggregate vs. getting the rows out (compare pandas_calc_s
        in day16_pandas.py):")
print(
    timing[["rows", "sql_agg_s", "read_sql_s",
        "read_sql/agg"]].to_string(
```

```
        index=False, float_format=lambda x: f"{x:,.4g}"
    )
)
print(
    "\nobservation: copying the table into a DataFrame costs more than
         the SQL aggregate:",
    bool((timing.read_sql_s > timing.sql_agg_s).all()),
)
```

Day 17 — Memory: lists, generators, DataFrames and chunks

Goal: measure how much memory the same rows cost as a list, as a stream and as a DataFrame; shrink the DataFrame with better dtypes; and aggregate a big table in memory proportional to a chunk, not to the table.

Read

Day 16 was about time. The other way a job fails is **memory**: the laptop swaps, or the container is killed. An FDD job over a year of 5-minute data on a hundred AHUs is tens of millions of rows. Ways to hold the rows:

Shape	What Python holds	Memory grows with
a `list` of tuples (`fetchall()`)	every row as a tuple of float objects	rows (and a lot of it per row)
a stream (`for row in cursor`, or `zip` over columns)	one tuple at a time	nothing: O(1)
a DataFrame (`read_csv`, `read_sql_query`)	NumPy arrays, one per column	rows x bytes per value
a chunked read (`chunksize=`)	one chunk of arrays at a time	the chunk size
a SQL aggregate	the answer (a couple of rows)	nothing: SQLite holds the data

A Python float is a 24-byte object, plus an 8-byte pointer in the tuple, plus the tuple's own header. A NumPy `float64` is 8 bytes, period. That's why DataFrames beat lists of tuples for numbers. But the DataFrame is still O(rows), and for text columns it's much worse: an `object` column stores a pointer per row to a separate Python string.

Two memory-saving moves, and one honest warning:

- **Right-size the dtypes.** `equipment_id` has 2 distinct values in 200,000 rows: store it as `category` (small integer codes plus one lookup table). A 0/1 status fits in `int8`. `float32` halves a column, but has only about 7 significant digits. That's fine for a temperature, risky for an epoch timestamp, and always something to *check* rather than assume.
- **Stream in chunks.** If the answer is an aggregate (counts, sums, a few rows per equipment), you never need the whole table in memory. Read a chunk, fold it into running totals, throw it away. Peak memory is O(chunk), and the result is the same as one big aggregate. Because sums and counts are *associative*, summing the chunk sums gives the total.

- **Warning:** `lab.peak_mib` uses `tracemalloc`, which sees memory allocated by Python and by NumPy/pandas buffers. It does **not** see memory SQLite allocates inside its own C library (page cache, sort buffers, the in-memory database itself). A SQL aggregate over 200,000 rows reports about zero because Python really did hold about nothing; SQLite's own footprint just isn't measured by it.

The task is the same as Day 16 per AHU: the count of raw FC1 faults (fan running, static more than 25 Pa below setpoint), plus run hours. The data is `lab.synthetic_history(100_000, equips=2)` (20,000 rows per AHU under the test): two AHUs, identical in both scripts.

Run

```
uv run python lessons/course.py run 17
```

That runs `day17_pandas.py`, then `day17_sql.py`. The plain-Python ways of holding rows live in the pandas script; the SQL script is the aggregate, plus the cost of pulling the rows out of SQLite.

Step	day17_pandas.py	day17_sql.py
reference answer	`hits(df).groupby(df.equipment_id).sum()`	`SUM(fan_status = 1 AND ... GROUP BY equipment`
rows as a list	`list(df.itertuples(index=False, name=None))`	`cursor.fetchall()` (trans only)
rows as a stream	`for ... in zip(*(df[c] for c in COLS))`	`for _ in cursor` (transfe only)
whole DataFrame	`pd.read_csv(path)`	`pd.read_sql_query(ROWS conn)`
smaller dtypes	`astype({"equipment_id": "category", "duct_static": "float32", ...})`	(SQLite stores what it stor
chunks	`pd.read_csv(path, chunksize=CHUNK)`, fold the partial sums	`read_sql_query(..., chunksize=CHUNK)` (transf only)
answers kept	`fault_counts`, `fault_hours`	`fault_counts`, `fault_hou`

The pandas script writes the synthetic rows to a temporary CSV, so its chunked read is a real file read. The SQL script's rows live in SQLite, so its chunked read comes from the database. Both print the per-AHU answer:

```
equipment_id  fault_rows  run_hours
       AHU_1        4993     4164.0
       AHU_2        5081     4164.0
```

Then the memory tables (MiB of Python heap at the peak; the numbers vary a little by Python version, but not by much):

```
        approach  peak_MiB             (day17_pandas.py)
  list of tuples     36.49
    stream (zip)      0.00
  read whole CSV     15.67
chunked CSV read      2.25

                    approach  peak_MiB (day17_sql.py)
SQL aggregate (2 rows back)     0.008
              fetchall list    45.414
              cursor stream     0.001
       read_sql_query whole    67.012
     read_sql_query chunked     6.189
```

Three things to see.

- **The stream is flat.** The loop holds one tuple, so its peak is about zero. Streaming trades speed for space (the interpreter loop is Day 16's slow one), but its memory cost is nothing.
- **A DataFrame's peak can be higher than its resting size.** `read_csv` parses straight into arrays (15.7 MiB). But `read_sql_query` has to fetch every row as Python tuples first and then convert them into column arrays, so for a moment both copies exist: 67 MiB, more than the 45 MiB list on its own. `chunksize` fixes that: 6 MiB, about a tenth of the whole read. Memory now scales with the **chunk**: double the table and the peak stays put (only the loop runs longer). The price is that this works for aggregates that can be folded (counts, sums, min/max). A rolling window or a streak that crosses a chunk boundary needs the chunks to carry state across, which is what Day 18's loop does naturally.
- **The SQL aggregate is the cheapest in Python memory**, 0.008 MiB: Python holds a 2-row result while SQLite scans every row. But that number is incomplete: it's the Python side. SQLite's own memory (the page cache, and here the entire in-memory database, 11.3 MiB as the second script prints from `PRAGMA page_count * page_size`) is invisible to `tracemalloc`. To see it you would watch the process, for example with `psutil`'s resident set size.

Shrinking the DataFrame. `memory_usage(deep=True)` counts the real bytes, including the Python objects behind an `object` column. The pandas script prints the table:

```
               before_MiB  after_MiB
equipment_id        10.30       0.19
ts                   1.53       1.53
fan_status           1.53       0.19
duct_static          1.53       0.76
duct_static_sp       1.53       0.76
total               16.40       3.43
```

The big win is `equipment_id`: an `object` column of strings becomes a column of 1-byte codes. The float columns halve, the flag drops 8x, and `ts` stays `int64` (epoch seconds need 32+ bits, and `float32` would corrupt them). `float32` changed the *values*, so the script checks that the fault decision survives: it prints `float32 flipped 0 of 200,000 fault decisions`. On this data no row flips: the readings carry two decimals and the threshold is a round number, so float32's rounding (about 3e-5 at 350 Pa) never crosses it. On data that sits *exactly on* a threshold, it could. Checking is how you find out, and why Open-FDD keeps one declared scale and dtype per column instead of guessing.

Try it

1. How big would the full DataFrame be for 100 AHUs x 1 year of 5-minute data (about 105,000 rows each) with five float64 columns plus an int64 `ts`? Compute it from the table above, no new data needed.
2. The chunked read in `day17_pandas.py` also returns run hours per AHU (`fan_status == 1` rows x 5 / 60), and `fault_hours` in `RESULTS` is compared with the SQL aggregate. Why is *run hours* easy to fold across chunks, and what about a median duct static?

Answers

1. Bytes per row of the numeric columns, times rows:

   ```
   numeric_bytes_per_row = 8 * 5 + 8  # five float64 + one int64
   rows_total = 100 * 105_000
   gib = rows_total * numeric_bytes_per_row / 2**30  # about 0.47 GiB
   ```

   About 0.5 GiB for 10.5 million rows: fine on a laptop, but tens of GiB across a portfolio of buildings. That is where chunking, dtypes and "let the database aggregate" stop being optional.

2. The partial results are small tables you can add: sums add. `faults_and_hours()` keeps a `fault_rows` and a `run_rows` column per chunk, adds the chunks' tables, and converts rows to hours at the end. A *mean* you'd carry as (sum, count) and divide at the end; a median doesn't fold this way at all.

Why it matters

Memory decides where a rule can run. A rule that needs the whole table as a DataFrame fits a laptop but not an edge controller (Normal Framework on a Raspberry Pi 3 has under 1 GB for everything). Streaming and chunking are what let the same logic run in both places. Day 20 puts time and memory side by side.

day17_pandas.py

```
"""Day 17, pandas side: how much memory the same rows cost as a list, a
        stream and a DataFrame.

    uv run python lessons/unit2-python/day17_pandas.py

The task for every version is the same: the count of raw FC1 faults
        (fan running, duct static more
than 25 Pa below setpoint) per AHU, plus run hours. Same synthetic rows
        as day17_sql.py (two AHUs).

The Python-side versions live here: a list of tuples, a lazy stream, a
        whole DataFrame, a DataFrame
with smaller dtypes, and a chunked read that never holds the whole
        table. Compare with day17_sql.py.
"""

import sys
import tempfile
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

TOL_PA = 25.0  # how far below setpoint counts as a fault
N = 20_000 if lab.QUICK else 100_000  # rows per AHU; two AHUs
CHUNK = N // 8  # rows per chunk when reading in pieces
COLS = ["equipment_id", "ts", "fan_status", "duct_static",
        "duct_static_sp"]

# 1. Data: the same synthetic history as the SQL script. We also write
        it to a CSV so we can show
#    a real chunked read (SQL's cursor has its own way, see
        day17_sql.py).
df = lab.synthetic_history(N, equips=2)[COLS]
tmp = tempfile.TemporaryDirectory(ignore_cleanup_errors=True)
csv_path = Path(tmp.name) / "h.csv"
df.to_csv(csv_path, index=False)
print(f"{len(df):,} rows")


# 2. The reference answer: the whole DataFrame in memory, one groupby
def hits(d, tol=TOL_PA):
    return (d.fan_status == 1) & (d.duct_static < d.duct_static_sp -
        tol)


fault_counts = (
    hits(df).groupby(df.equipment_id).sum().astype("int64").rename("fault_rows")
)
print(fault_counts.to_string(index=False))
```

```
# 3. Three ways to hold the rows. Each returns the same per-AHU counts;
        peak_mib measures the
#    Python-heap peak while it runs.
def counts_list():
    rows = list(df.itertuples(index=False, name=None))  # a list of N
        tuples, all alive at once
    out = {}
    for eq, _ts, fan, static, sp in rows:
        out[eq] = out.get(eq, 0) + (1 if fan == 1 and static < sp -
            TOL_PA else 0)
    return out


def counts_stream():
    out = {}
    for eq, _ts, fan, static, sp in zip(*(df[c] for c in COLS),
            strict=True):  # one row at a time
        out[eq] = out.get(eq, 0) + (1 if fan == 1 and static < sp -
            TOL_PA else 0)
    return out


def counts_dataframe():
    d = pd.read_csv(csv_path)  # the whole file as NumPy arrays
    return hits(d).groupby(d.equipment_id).sum().to_dict()


approaches = {
    "list of tuples": counts_list,
    "stream (zip)": counts_stream,
    "read whole CSV": counts_dataframe,
}
mem_rows, agree = [], {}
for name, fn in approaches.items():
    got = fn()
    agree[name] = {k: int(v) for k, v in got.items()} == dict(
        zip(fault_counts.equipment_id, fault_counts.fault_rows,
            strict=True)
    )
    mem_rows.append({"approach": name, "peak_MiB": lab.peak_mib(fn)})


# 4. Shrinking the DataFrame. memory_usage(deep=True) counts the real
        bytes, including the Python
#    objects behind an `object` column.
d = pd.read_csv(csv_path)
before = d.memory_usage(deep=True)
small = d.astype(
    {
        "equipment_id": "category",  # 2 distinct values -> 1-byte
            codes plus a lookup table
        "fan_status": "int8",  # a 0/1 flag
        "duct_static": "float32",
        "duct_static_sp": "float32",
    }
)  # ts stays int64: epoch seconds (~1.8e9) need 32+ bits, and float32
        would corrupt them
after = small.memory_usage(deep=True)
```

```
sizes = pd.DataFrame({"before_MiB": before / 2**20, "after_MiB": after
        / 2**20}).drop(index="Index")
sizes.loc["total"] = sizes.sum()
hit64 = hits(d)
hit32 = (small.fan_status == 1) & (small.duct_static <
        small.duct_static_sp - np.float32(TOL_PA))
flips = int((hit64 != hit32).sum())


# 5. Chunked read: fold each chunk into running totals, never hold the
        whole table.
def faults_and_hours():
    total = None
    for chunk in pd.read_csv(csv_path, chunksize=CHUNK):  # an iterator
        of DataFrames
        part = (
            pd.DataFrame(
                {
                    "fault_rows": hits(chunk).astype(int),
                    "run_rows": (chunk.fan_status == 1).astype(int),
                }
            )
            .groupby(chunk.equipment_id)
            .sum()
        )
        total = part if total is None else total.add(part,
         fill_value=0)  # sums are associative
    total["run_hours"] = total.run_rows * 5 / 60
    total["fault_hours"] = total.fault_rows.astype("int64")
    return total.reset_index()[["equipment_id", "fault_hours",
        "run_hours"]]


fault_hours = faults_and_hours()
mem_rows.append({"approach": "chunked CSV read", "peak_MiB":
        lab.peak_mib(faults_and_hours)})
mem_table = pd.DataFrame(mem_rows)

RESULTS = {"fault_counts": fault_counts, "fault_hours": fault_hours}

print("\nfault rows and run hours per AHU (from the chunked read):")
print(fault_hours.round(3).to_string(index=False))
print("\nPeak Python-heap memory while computing the same per-AHU
        counts:")
print(mem_table.to_string(index=False, float_format=lambda x: f"
        {x:,.2f}"))
print("\nDataFrame resident size, default dtypes vs smaller dtypes:")
print(sizes.round(2).to_string())
print("\nchecks (printed, not asserted):")
print("  every approach gives the reference counts:", agree)
print(
    "  chunked counts equal the whole-table counts:",
    fault_hours.fault_rows.tolist() ==
        fault_counts.fault_rows.tolist(),
)
print(f"  float32 flipped {flips} of {len(d):,} fault decisions")
print(
    f"  mean static, float64 vs float32: {d.duct_static.mean():.6f}
        {small.duct_static.mean():.6f}"
```

```
)
peak = mem_table.set_index("approach").peak_MiB
print(f"  stream peak {peak['stream (zip)']:.2f} MiB vs list
        {peak['list of tuples']:.2f} MiB")
print(f"  chunked peak {peak['chunked CSV read']:.2f} MiB, whole
        {peak['read whole CSV']:.2f} MiB")
tmp.cleanup()
```

day17_sql.py

```
"""Day 17, SQL side: the same per-AHU counts as aggregates, and what
        pulling the rows out costs.

    uv run python lessons/unit2-python/day17_sql.py

The task for every version is the same: the count of raw FC1 faults
        (fan running, duct static more
than 25 Pa below setpoint) per AHU, plus run hours. Same synthetic rows
        as day17_pandas.py, loaded
into an in-memory SQLite table first (setup). SQLite does the
        computation; Python holds 2 rows.

The memory comparison measures the Python heap only (tracemalloc). It
        cannot see SQLite's own
memory, so we also print the size of the database itself. Compare with
        day17_pandas.py.
"""

import sqlite3
import sys
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

PARAMS = {"tol": 25.0}  # how far below setpoint counts as a fault
N = 20_000 if lab.QUICK else 100_000  # rows per AHU; two AHUs
CHUNK = N // 8  # rows per chunk when pulling rows out in pieces
ROWS_SQL = "SELECT equipment_id, ts, fan_status, duct_static,
        duct_static_sp FROM h"

COUNTS_SQL = """
SELECT equipment_id,
       SUM(fan_status = 1 AND duct_static < duct_static_sp - :tol) AS
         fault_rows
FROM h GROUP BY equipment_id ORDER BY equipment_id
"""

HOURS_SQL = """
SELECT equipment_id,
       SUM(fan_status = 1 AND duct_static < duct_static_sp - :tol) AS
         fault_rows,
       SUM(fan_status = 1) * 5 / 60.0 AS run_hours
FROM h GROUP BY equipment_id ORDER BY equipment_id
"""

# 1. Data: the same synthetic history as the pandas script, loaded into
        SQLite (setup)
conn = sqlite3.connect(":memory:")
lab.to_sqlite(lab.synthetic_history(N, equips=2), conn, "h", index=
        ["equipment_id", "ts"])
print(f"{conn.execute('SELECT count(*) FROM h').fetchone()[0]:,} rows
        in table h")
```

```
# 2. The answers: one GROUP BY each. SQLite scans every row; Python
        receives two rows.
fault_counts = pd.read_sql_query(COUNTS_SQL, conn, params=PARAMS)
fault_hours = pd.read_sql_query(HOURS_SQL, conn, params=PARAMS)
print(fault_counts.to_string(index=False))


# 3. Peak Python-heap memory: the aggregate against pulling the rows
        out (transfer only, no maths)
def sql_aggregate():
    return pd.read_sql_query(COUNTS_SQL, conn, params=PARAMS)


def pull_fetchall():
    return len(conn.execute(ROWS_SQL).fetchall())  # a list of N
        tuples, all alive at once


def pull_stream():
    return sum(1 for _ in conn.execute(ROWS_SQL))  # one tuple at a
        time


def pull_dataframe():
    return len(pd.read_sql_query(ROWS_SQL, conn))  # whole result as
        NumPy arrays


def pull_chunked():
    return sum(len(c) for c in pd.read_sql_query(ROWS_SQL, conn,
        chunksize=CHUNK))


mem_table = pd.DataFrame(
    [
        {"approach": name, "peak_MiB": lab.peak_mib(fn)}
        for name, fn in {
            "SQL aggregate (2 rows back)": sql_aggregate,
            "fetchall list": pull_fetchall,
            "cursor stream": pull_stream,
            "read_sql_query whole": pull_dataframe,
            "read_sql_query chunked": pull_chunked,
        }.items()
    ]
)

# 4. What tracemalloc cannot see: the in-memory database itself
page_count = conn.execute("PRAGMA page_count").fetchone()[0]
page_size = conn.execute("PRAGMA page_size").fetchone()[0]
db_mib = page_count * page_size / 2**20

RESULTS = {"fault_counts": fault_counts, "fault_hours": fault_hours}

print("\nfault rows and run hours per AHU:")
print(fault_hours.round(3).to_string(index=False))
print("\nPeak Python-heap memory (the rows are pulled out but not
        computed on):")
```

```
print(mem_table.to_string(index=False, float_format=lambda x: f"
        {x:,.3f}"))
print(f"\nSQLite's own memory, which tracemalloc cannot see: the
        database is {db_mib:.1f} MiB")
peak = mem_table.set_index("approach").peak_MiB
print()
print("observation: peak MiB by approach:", peak.round(3).to_dict())
```

Day 18 — Gaps and coverage: sample-and-hold, three ways

Goal: make a fault rule honest when the data has holes: hold each reading until the next one, treat a long gap as "unknown", confirm faults by elapsed time rather than row count, and report coverage next to fault hours. Implement it in SQL, pandas and plain Python, and compare all three.

Read

Day 14 counted rows and multiplied by 5 minutes. That works only if every 5-minute row exists. Real controllers go offline, networks drop, a trend log gets cleared. Open-FDD's rules (read `sql/fdd/postgres/rules/cmd1_fan_mismatch.sql`) handle this with **forward sample-and-hold**:

- A reading is assumed to hold until the *next reading arrives*. `held_seconds = next_ts - ts`. A missing row just means its predecessor holds for 10 minutes instead of 5, and the time is still counted.
- If the next reading is more than `MAX_GAP_HOURS` away, we don't know what happened. `held_seconds = 0`: that stretch is **not fault time, not covered time**, and `gap_after = 1` marks the row so the streak **breaks** there. The last row of each equipment has no next reading, so `held_seconds = 0`.
- A fault is **confirmed** when its streak has lasted `CONFIRM_SECONDS` *of elapsed time*: `streak_elapsed = running sum of held_seconds within the streak`.
- Report `fault_hours = sum(held_seconds where confirmed) / 3600` and **coverage**: the seconds we could vouch for divided by the seconds in the window (first to last timestamp).

Why counting rows is wrong once there are gaps. Three things go wrong. (1) A dropped row silently removes 5 minutes of fault time. (2) Two fault rows on either side of a six-hour outage look like consecutive rows, so the streak "confirms" across time you never observed. (3) You cannot tell a healthy machine from one you couldn't see: `fault_hours = 0` with 40% coverage is not good news. Coverage keeps the answer honest.

The data is synthetic (`lab.synthetic_history(4000, equips=2, gap_prob=0.03)`): two AHUs with 3% of rows dropped at random (short gaps of 10-15 minutes), plus one long outage on AHU_1: 100 rows (about 8 hours) deleted from inside its weak-fan window, while the fan is on. Both scripts build exactly the same table (7,648 of 8,000 rows). The constants are at the top of each script: `MAX_GAP_HOURS = 1`, `CONFIRM_SECONDS = 600` (the same 10 minutes as Day 14's two rows).

How each step maps across (compare Day 14's table):

Step	`day18_sql.py`	`day18_pandas.py` (vectorized)	`day18_pandas.py` (plain Python loop)
next timestamp	`LEAD(ts) OVER (PARTITION BY eq ORDER BY ts)`	`df.groupby(eq).ts.shift(-1)`	look at the next row if same equipment
held / gap flag	`CASE WHEN dt > :max_gap_s THEN ...`	`np.where(...)`	`if dt > limit:`
streak breaks	`LAG(raw)` and `LAG(gap_after)` in a `CASE`	`raw.ne(raw.shift())` or previous gap	`if raw != prev_raw or prev_gap:`
elapsed in streak	`SUM(held) OVER (PARTITION BY eq, streak_id ORDER BY ts)`	`held.groupby([eq, streak]).cumsum()`	`elapsed += held`

SQLite has no `EXTRACT(EPOCH FROM ...)` like Postgres, but `ts` is already epoch seconds, so `next_ts - ts` is seconds directly. Units are SI: Pa, and seconds for time.

Run

```
uv run python lessons/course.py run 18
```

That runs `day18_pandas.py` (vectorized pandas, then the plain Python loop in its own section 3), then `day18_sql.py`. `RESULTS` in both holds the per-row state (`rows`), the per-AHU `summary`, the row-counting comparison (`naive_fault_hours`) and the two Try-it variants. Both print the summary:

```
equipment_id  rows  long_gaps  fault_hours  covered_hours  window_hours
coverage
       AHU_1  3785          1        6.750        324.833        333.25
0.975
       AHU_2  3863          0       14.417        333.250        333.25
1.000
```

Look at what the gaps did. AHU_2 has only short drops, and sample-and-hold covers them: coverage 1.0. AHU_1's 8-hour outage is longer than `MAX_GAP_HOURS`, so it is `long_gaps = 1` and coverage falls to 0.975 (covered time = window time minus the long gaps; the pandas script prints that check). The one row with `gap_after = 1` has `held_seconds` 0 and the next row starts a new streak, so the fault episode that runs into the outage stops at the outage rather than stretching across it.

The pandas script also checks the loop against the vectorized version, and prints `loop rows == pandas rows: True` and `loop summary == pandas summary: True`. The `streak_id` of each row and the `streak_elapsed` are in `rows` for a row-by-row comparison with the SQL.

Now the wrong way. Day 14's row-counting version, run on the same gappy rows (section 4 in both scripts):

```
equipment_id  naive_fault_hours  fault_hours  difference_h
       AHU_1               6.67         6.75          0.08
       AHU_2              13.92        14.42          0.50
```

AHU_2 has only short drops, and the row-count version is 0.5 hours low: every dropped row took 5 minutes of fault time with it, where sample-and-hold keeps them. AHU_1 is off by much less, but not because the row-count method is right there. It loses time to dropped rows *and* has no notion of the outage (the sample-and-hold version breaks the streak at the outage and counts nothing inside it), so its errors partly cancel by luck. Only the gap-aware numbers come with a coverage figure that tells you how far to trust them.

Where the plain loop wins. The loop is one forward pass with four pieces of state (the previous raw flag, whether the previous row ended in a gap, the streak number, the running elapsed time), and it reads like the description in "Read". SQL needs four window functions and a CTE per step for the same thing. The loop has to reset its state when the equipment changes (`if eq != prev_eq`); `PARTITION BY` does that for SQL for free.

Try it

1. Raise `MAX_GAP_HOURS` to 12. What happens to AHU_1's coverage, its `gap_after` count and its fault hours?
2. Confirm faults over 1 hour (`CONFIRM_SECONDS = 3600`) instead of 10 minutes. Do fault hours rise or fall, and why?

Both are already computed in each script's `RESULTS` (`summary_12h_limit`, `summary_confirm_1h`), so the pandas and SQL answers are compared. Change the constants at the top of a script to see them from a different angle.

Answers

1. The printout shows `long_gaps` 0 and coverage 1.0 for both AHUs, and AHU_1's fault hours jump from 6.75 to 15.17: it now counts the 8 hours *as if the fault state at the start of the outage persisted through it*. That is the cost of a large `MAX_GAP_HOURS`: you assert knowledge you don't have. Pick the limit from how fast the equipment can change state, not from how much you'd like to cover.
2. Fault hours fall (AHU_1 6.75 to 5.08 h, AHU_2 14.42 to 12.75 h). Faults shorter than an hour never confirm, and in longer ones the first hour of

each episode is not counted (a row only counts once its streak's elapsed time reaches the threshold). A longer confirmation time trades sensitivity for fewer false alarms.

Why it matters

Every real Open-FDD rule has this `elapsed_hold` block, and its parity tests check it. A rule without it will pass on clean test data and quietly over- or under-report on a building with a flaky gateway. Day 19 joins a second time series (weather) to these rows, and the same question (how old is too old?) returns.

day18_pandas.py

```
"""Day 18, pandas side: FC1 with sample-and-hold, a gap limit and
        coverage, on data with holes.

    uv run python lessons/unit2-python/day18_pandas.py

Day 14 counted rows and multiplied by 5 minutes. With dropped rows that
        is wrong. Here each reading
holds until the next one arrives (`held_seconds = next_ts - ts`), a gap
        longer than MAX_GAP_HOURS is
"unknown" (held 0, the streak breaks), a fault confirms after
        CONFIRM_SECONDS of elapsed time, and
coverage says how much of the window we could vouch for.

This script has the vectorized pandas version and the plain Python loop
        (one forward pass with four
pieces of state). Both run on the same synthetic gappy rows as
        day18_sql.py.
"""

import contextlib
import io
import sys
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

TOL_PA = 25.0  # how far below setpoint counts as a fault
MAX_GAP_HOURS = 1.0  # a longer silence is "unknown", not "still
        faulty"
CONFIRM_SECONDS = 600  # 10 minutes, the same as Day 14's two 5-minute
        rows
N = 4_000  # rows per AHU before drops: about 14 days

# 1. Data with holes: 3% of rows dropped at random (short gaps of 10-15
        minutes), plus one long
#    outage on AHU_1: 100 rows (about 8 hours) deleted from inside its
        weak-fan window.
raw_df = lab.synthetic_history(N, equips=2, gap_prob=0.03)
T0 = int(raw_df.ts.min())
outage = (raw_df.equipment_id == "AHU_1") & raw_df.ts.between(T0 + 1560
        * 300, T0 + 1659 * 300)
gappy = raw_df[~outage].sort_values(["equipment_id",
        "ts"]).reset_index(drop=True)
steps = gappy.groupby("equipment_id").ts.diff().dropna()
print(f"{len(gappy):,} rows (of {2 * N:,}); seconds between rows:")
print(steps.value_counts().sort_index().to_dict())

COLUMNS = ["equipment_id", "ts", "raw_fault", "held_seconds",
        "gap_after", "streak_id"]
COLUMNS += ["streak_elapsed", "confirmed"]
```

```python
# 2. pandas: every step a column operation. shift(-1) inside a groupby
        is LEAD; shift() is LAG.
def gap_pandas(df, max_gap_h=MAX_GAP_HOURS, confirm_s=CONFIRM_SECONDS,
        tol=TOL_PA):
    """df sorted by (equipment_id, ts). Returns one row of state per
        input row."""
    eq = df["equipment_id"]
    raw = ((df.fan_status == 1) & (df.duct_static < df.duct_static_sp -
        tol)).astype(int)
    dt = df.groupby("equipment_id").ts.shift(-1) - df.ts  # NaN on each
        equipment's last row
    limit = max_gap_h * 3600.0
    held = pd.Series(np.where((dt > 0) & (dt <= limit), dt, 0.0),
        index=df.index)  # NaN -> 0
    gap_after = (dt > limit).astype(int)
    prev_gap = gap_after.groupby(eq).shift().fillna(0)
    same = (raw == raw.groupby(eq).shift()) & (prev_gap == 0)  # first
        row: NaN -> False -> new
    streak_id = (~same).astype(int).groupby(eq).cumsum()
    elapsed = held.groupby([eq, streak_id]).cumsum()
    confirmed = ((raw == 1) & (elapsed >= confirm_s)).astype(int)
    return df[["equipment_id", "ts"]].assign(
        raw_fault=raw,
        held_seconds=held,
        gap_after=gap_after,
        streak_id=streak_id,
        streak_elapsed=elapsed,
        confirmed=confirmed,
    )


def summarize(rows):
    g = rows.assign(fault_s=rows.held_seconds.where(rows.confirmed ==
        1, 0.0)).groupby(
        "equipment_id"
    )
    out = g.agg(
        rows=("ts", "size"),
        long_gaps=("gap_after", "sum"),
        fault_s=("fault_s", "sum"),
        covered_s=("held_seconds", "sum"),
        t0=("ts", "min"),
        t1=("ts", "max"),
    )
    out["fault_hours"] = out.fault_s / 3600.0
    out["covered_hours"] = out.covered_s / 3600.0
    out["window_hours"] = (out.t1 - out.t0) / 3600.0
    out["coverage"] = out.covered_s / (out.t1 - out.t0)
    cols = ["rows", "long_gaps", "fault_hours", "covered_hours",
        "window_hours", "coverage"]
    return out[cols].reset_index()


rows_pd = gap_pandas(gappy)
summary = summarize(rows_pd)


# 3. Plain Python: a single forward pass. State: previous raw flag,
        whether the previous row ended
```

```python
#    in a gap, the streak number and the running elapsed time. The only
        look-ahead is the next
#    row's timestamp.
def gap_loop(df, max_gap_h=MAX_GAP_HOURS, confirm_s=CONFIRM_SECONDS,
        tol=TOL_PA):
    data = list(
        df[["equipment_id", "ts", "fan_status", "duct_static",
         "duct_static_sp"]].itertuples(
            index=False, name=None
        )
    )
    limit = max_gap_h * 3600.0
    out, totals = [], {}
    prev_eq = prev_raw = prev_gap = None
    streak_id = 0
    elapsed = 0.0
    for i, (eq, ts, fan, static, sp) in enumerate(data):
        raw = 1 if fan == 1 and static < sp - tol else 0
        has_next = i + 1 < len(data) and data[i + 1][0] == eq
        dt = data[i + 1][1] - ts if has_next else None
        held = dt if dt is not None and 0 < dt <= limit else 0.0
        gap = 1 if dt is not None and dt > limit else 0
        if eq != prev_eq:  # a new equipment: reset the state machine
            streak_id, elapsed, prev_raw, prev_gap = 0, 0.0, None, 0
            totals[eq] = {"rows": 0, "long_gaps": 0, "fault_s": 0.0,
         "covered_s": 0.0, "t0": ts}
        if raw != prev_raw or prev_gap:
            streak_id += 1
            elapsed = 0.0
        elapsed += held
        confirmed = 1 if raw == 1 and elapsed >= confirm_s else 0
        out.append((eq, ts, raw, held, gap, streak_id, elapsed,
         confirmed))
        t = totals[eq]
        t["rows"] += 1
        t["long_gaps"] += gap
        t["covered_s"] += held
        t["fault_s"] += held if confirmed else 0.0
        t["t1"] = ts
        prev_eq, prev_raw, prev_gap = eq, raw, gap
    rows = pd.DataFrame(out, columns=COLUMNS)
    summary = pd.DataFrame(
        [
            {
                "equipment_id": eq,
                "rows": t["rows"],
                "long_gaps": t["long_gaps"],
                "fault_hours": t["fault_s"] / 3600.0,
                "covered_hours": t["covered_s"] / 3600.0,
                "window_hours": (t["t1"] - t["t0"]) / 3600.0,
                "coverage": t["covered_s"] / (t["t1"] - t["t0"]),
            }
            for eq, t in totals.items()
        ]
    )
    return rows, summary
```

```
rows_loop, summary_loop = gap_loop(gappy)


def same(a, b):
    """True if two frames hold the same values (row order and dtypes
        ignored)."""
    try:
        with contextlib.redirect_stdout(io.StringIO()):  #
         assert_parity prints on success
            lab.assert_parity(a, b, label="")
    except AssertionError:
        return False
    return True


# 4. The wrong way: Day 14's row-counting version on the same gappy
        rows
raw = ((gappy.fan_status == 1) & (gappy.duct_static <
        gappy.duct_static_sp - TOL_PA)).astype(int)
eq = gappy.equipment_id
streak = raw.ne(raw.groupby(eq).shift()).astype(int).groupby(eq).cumsum()
length = raw.groupby([eq, streak]).cumcount() + 1
naive_rows = ((raw == 1) & (length >= 2)).groupby(eq).sum()
naive = (naive_rows * 5 /
        60.0).rename("naive_fault_hours").reset_index()

# 5. Try-it variants: a 12-hour gap limit, and confirming over 1 hour
summary_12h = summarize(gap_pandas(gappy, max_gap_h=12.0))
summary_confirm_1h = summarize(gap_pandas(gappy, confirm_s=3600))

RESULTS = {
    "rows": rows_pd,
    "summary": summary,
    "naive_fault_hours": naive,
    "summary_12h_limit": summary_12h,
    "summary_confirm_1h": summary_confirm_1h,
}

print("\nGap-aware summary (loop and pandas give the same table):")
print(summary.round(3).to_string(index=False))
print("\nThe one row with gap_after = 1 per AHU that has an outage:")
print(rows_pd[rows_pd.gap_after == 1].round(1).to_string(index=False))

cmp = naive.merge(summary[["equipment_id", "fault_hours"]],
        on="equipment_id")
cmp["difference_h"] = cmp.fault_hours - cmp.naive_fault_hours
print("\nRow counting (Day 14) vs gap-aware, fault hours:")
print(cmp.round(2).to_string(index=False))

print("\nTry-it variants (fault hours / coverage):")
print("MAX_GAP_HOURS = 12:")
print(
    summary_12h[["equipment_id", "long_gaps", "fault_hours",
        "coverage"]]
    .round(3)
    .to_string(index=False)
)
print("CONFIRM_SECONDS = 3600:")
```

```
print(summary_confirm_1h[["equipment_id",
         "fault_hours"]].round(2).to_string(index=False))

print("\nchecks (printed, not asserted):")
print("  loop rows == pandas rows:", same(rows_loop, rows_pd))
print("  loop summary == pandas summary:", same(summary_loop, summary))
s = summary.set_index("equipment_id")
print("  AHU_2 (short drops only) coverage:", round(s.loc["AHU_2",
         "coverage"], 6))
print("  AHU_1 (with the 8 h outage) coverage:", round(s.loc["AHU_1",
         "coverage"], 6))
long_gap_s = steps[steps > MAX_GAP_HOURS *
         3600].groupby(gappy.equipment_id).sum()
window_s = gappy.groupby("equipment_id").ts.agg(lambda x: x.max() -
         x.min())
print(
    "  AHU_1 covered hours:",
    round(s.loc["AHU_1", "covered_hours"], 3),
    "= window minus long gaps:",
    round((window_s["AHU_1"] - long_gap_s["AHU_1"]) / 3600.0, 3),
)
```

day18_sql.py

```
"""Day 18, SQL side: FC1 with forward sample-and-hold, a gap limit and
        coverage, on data with holes.

    uv run python lessons/unit2-python/day18_sql.py

The same computation as day18_pandas.py on the same synthetic gappy
        rows (loaded into in-memory
SQLite as setup). Same shape as Open-FDD's cmd1_fan_mismatch.sql, in
        SQLite's dialect:
LEAD gives the next timestamp, so `held_seconds = next_ts - ts`; a gap
        longer than MAX_GAP_HOURS is
held 0 and breaks the streak; a fault confirms after CONFIRM_SECONDS of
        elapsed time (a running SUM
of held_seconds inside the streak); coverage is held seconds over the
        window.
"""

import sqlite3
import sys
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

PARAMS = {
    "tol": 25.0,  # how far below setpoint counts as a fault, pascals
    "max_gap_s": 1.0 * 3600,  # MAX_GAP_HOURS = 1: a longer silence is
        "unknown"
    "confirm_s": 600,  # 10 minutes, the same as Day 14's two 5-minute
        rows
}
N = 4_000  # rows per AHU before drops: about 14 days

# 1. Data with holes (same recipe as the pandas script): 3% of rows
        dropped at random, plus one
#    long outage on AHU_1 (100 rows, about 8 hours) inside its weak-fan
        window. Setup.
raw_df = lab.synthetic_history(N, equips=2, gap_prob=0.03)
T0 = int(raw_df.ts.min())
outage = (raw_df.equipment_id == "AHU_1") & raw_df.ts.between(T0 + 1560
        * 300, T0 + 1659 * 300)
gappy = raw_df[~outage].sort_values(["equipment_id",
        "ts"]).reset_index(drop=True)
conn = sqlite3.connect(":memory:")
lab.to_sqlite(gappy, conn, "gap_h", index=["equipment_id", "ts"])
print(conn.execute("SELECT count(*) FROM gap_h").fetchone()[0], "rows
        in gap_h")

# 2. One query returns every row's state; one CTE per step.
ROW_SQL = """
WITH base AS (                                  -- gate + raw fault
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
         :tol
```

```
            THEN 1 ELSE 0 END AS raw_fault
  FROM gap_h
), stepped AS (                                -- LEAD: seconds until
        the next reading
  SELECT *, LEAD(ts) OVER (PARTITION BY equipment_id ORDER BY ts) - ts
        AS dt_seconds
  FROM base
), elapsed_hold AS (                           -- sample-and-hold, with
        the gap limit
  SELECT *,
        CASE WHEN dt_seconds IS NULL OR dt_seconds <= 0 OR dt_seconds
        > :max_gap_s
             THEN 0.0 ELSE dt_seconds END AS held_seconds,
        CASE WHEN dt_seconds > :max_gap_s THEN 1 ELSE 0 END AS
        gap_after
  FROM stepped
), lagged AS (                                 -- a streak breaks on a
        change or after a gap
  SELECT *,
        CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
        equipment_id ORDER BY ts)
                  AND COALESCE(LAG(gap_after) OVER (PARTITION BY
        equipment_id ORDER BY ts), 0) = 0
             THEN 0 ELSE 1 END AS is_new_streak
  FROM elapsed_hold
), grp AS (
  SELECT *, SUM(is_new_streak) OVER (PARTITION BY equipment_id ORDER BY
        ts
                                    ROWS UNBOUNDED PRECEDING) AS
        streak_id
  FROM lagged
), ranked AS (                                 -- elapsed time inside
        each streak
  SELECT *, SUM(held_seconds) OVER (PARTITION BY equipment_id,
        streak_id ORDER BY ts
                                   ROWS UNBOUNDED PRECEDING) AS
        streak_elapsed
  FROM grp
)
SELECT equipment_id, ts, raw_fault, held_seconds, gap_after, streak_id,
        streak_elapsed,
       CASE WHEN raw_fault = 1 AND streak_elapsed >= :confirm_s THEN 1
        ELSE 0 END AS confirmed
FROM ranked
"""

# 3. Summary: fault hours, covered hours and coverage per AHU
SUMMARY_SQL = f"""
SELECT equipment_id,
       COUNT(*) AS rows,
       SUM(gap_after) AS long_gaps,
       SUM(CASE WHEN confirmed = 1 THEN held_seconds ELSE 0.0 END) /
        3600.0 AS fault_hours,
       SUM(held_seconds) / 3600.0 AS covered_hours,
       (MAX(ts) - MIN(ts)) / 3600.0 AS window_hours,
       SUM(held_seconds) / (MAX(ts) - MIN(ts)) AS coverage
FROM ({ROW_SQL})
GROUP BY equipment_id
ORDER BY equipment_id
"""
```

```
# 4. The wrong way: Day 14's row-counting version on the same gappy
        table
NAIVE_SQL = """
WITH base AS (
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
         :tol THEN 1 ELSE 0 END
           AS raw_fault
  FROM gap_h
), marks AS (
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts)
                THEN 0 ELSE 1 END AS new_streak
  FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
         ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id
         ORDER BY ts) AS streak_len
  FROM streaks
)
SELECT equipment_id, SUM(raw_fault = 1 AND streak_len >= 2) * 5 / 60.0
         AS naive_fault_hours
FROM ranked GROUP BY equipment_id ORDER BY equipment_id
"""


def run(sql, **override):
    return pd.read_sql_query(sql, conn, params={**PARAMS, **override})


rows = run(ROW_SQL)
summary = run(SUMMARY_SQL)
naive = run(NAIVE_SQL)

# 5. Try-it variants: a 12-hour gap limit, and confirming over 1 hour
summary_12h = run(SUMMARY_SQL, max_gap_s=12.0 * 3600)
summary_confirm_1h = run(SUMMARY_SQL, confirm_s=3600)

RESULTS = {
    "rows": rows,
    "summary": summary,
    "naive_fault_hours": naive,
    "summary_12h_limit": summary_12h,
    "summary_confirm_1h": summary_confirm_1h,
}

print("\nGap-aware summary:")
print(summary.round(3).to_string(index=False))
print("\nThe one row with gap_after = 1 per AHU that has an outage:")
print(rows[rows.gap_after == 1].round(1).to_string(index=False))

cmp = naive.merge(summary[["equipment_id", "fault_hours"]],
        on="equipment_id")
cmp["difference_h"] = cmp.fault_hours - cmp.naive_fault_hours
print("\nRow counting (Day 14) vs gap-aware, fault hours:")
```

```
print(cmp.round(2).to_string(index=False))

print("\nTry-it variants (fault hours / coverage):")
print("MAX_GAP_HOURS = 12:")
print(
    summary_12h[["equipment_id", "long_gaps", "fault_hours",
         "coverage"]]
    .round(3)
    .to_string(index=False)
)
print("CONFIRM_SECONDS = 3600:")
print(summary_confirm_1h[["equipment_id",
         "fault_hours"]].round(2).to_string(index=False))
```

Day 19 — Joining weather: the as-of join

Goal: attach the most recent weather reading to every 5-minute AHU row, ignoring readings older than two hours, in SQL (correlated subquery plus an index) and in pandas (`merge_asof`); see why the three ways to do it scale so differently; then write a sensor-versus-weather rule on top.

Read

Some Open-FDD rules need a number the building doesn't measure well: the real outdoor temperature. A sensor on an AHU's outside-air intake can sit in the sun, in a shaded louver, or be wired wrong. The **`OAT-METEO`** rule compares it with a weather feed (Open-FDD pulls hourly data from Open-Meteo) and flags a sensor that disagrees with the weather.

The join is awkward because the two series don't share timestamps. The AHU logs every 5 minutes; the weather arrives hourly and sometimes not at all. You can't `JOIN ... ON h.ts = w.ts`. You want, for each AHU row, **the latest weather row at or before that time, but only if it is recent enough** (here: no more than 2 hours old; otherwise it is "stale", and the answer is `NULL`/`NaN`, meaning unknown, not "use whatever's old"). This is an **as-of join**.

The data. The AHU side is the real Building 50 export (both CSVs, 71,072 rows, `supply_fan_status` and `outside_air_temp_f` converted to °C: `(f - 32) * 5 / 9`, since fault math is SI). **The weather is a stand-in.** There is no weather feed in this repo, so we make one honestly: hourly means of AHU_1's own outdoor sensor, plus 0.5 °C of noise (real weather never matches a sensor exactly), with 12% of the hours deleted and one 6-hour block missing, the way a feed with failures looks (2,592 of 2,961 hours remain). In production this table is the Open-Meteo feed. It is derived from AHU_1, so AHU_1 will agree with it; AHU_2 has its own sensor, which reads about 1.2 °C higher on average and has a few readings of 0 °F, which look like dropouts. The rule should notice.

The weather stand-in is made by the same small `make_weather` function in both scripts (pandas builds it, the SQL script loads it into a `wx` table): it is *input data*, not the computation we compare.

How the join is done in each tool:

	SQL (`day19_sql.py`)	**pandas (`day19_pandas.py`)**	**plain Python (`day19_pandas.py`)**
Idea	per AHU row, look up the newest weather row `<= ts`	merge two sorted tables in one pass	per AHU row, search the weather list

	SQL (day19_sql.py)	pandas (day19_pandas.py)	plain Python (day19_pandas.py)
Code	(SELECT w.ts FROM wx w WHERE w.ts <= h.ts AND w.ts >= h.ts - :tol ORDER BY w.ts DESC LIMIT 1)	pd.merge_asof(h, wx, on="ts", direction="backward", tolerance=...)	bisect_right(wx_ts, ts) - 1
Stale handling	extra AND w.ts >= h.ts - :tol	tolerance= argument	if ts - wx_ts > tol

Complexity. Let N be AHU rows and M weather rows.

- A *nested loop* with no index: for each of N rows, scan all M weather rows. **O(N·M)**. SQLite does this when there is no index on `wx(ts)`; so does a naive Python double loop.
- *With an index* (SQLite) or a sorted list and binary search (Python's `bisect`): each lookup is O(log M), so **O(N log M)**.
- `merge_asof` on two *sorted* inputs walks both once: **O(N + M)**. The price: the inputs must be sorted by the key, which costs O(N log N) if they aren't.

Run

```
uv run python lessons/course.py run 19
```

That runs `day19_pandas.py`, then `day19_sql.py`. `ts` is epoch seconds in both (`unixepoch(timestamp_utc)` in SQL, `pd.to_datetime(...)` as seconds in pandas). Both build the per-row join, the rule, the 1-hour variant and a growing-size benchmark on synthetic rows (`lab.synthetic_history`, one AHU, weather one hour per 12 rows). `RESULTS` holds the per-row join (`asof_join`), the rule's `oat_meteo_hours`, `rows_without_weather` and the number of matches per benchmark size (`timing_matches`).

```
71,072 AHU rows, 2,592 weather rows (of 2,961 hours)
rows with no usable weather: 1250
```

The as-of join in SQL. The inner query is run once *per AHU row*, which is what makes it "correlated". The weather value is then looked up by that timestamp. In pandas, `merge_asof` needs both sides sorted by the key. Our table has two AHUs interleaved by time, so we sort by `ts` for the merge, and copy `ts` into a `wx_ts` column so we can see which weather row matched. A detail worth knowing: both treat the 2-hour limit as inclusive, so a reading exactly 7,200 s old still counts. (The unit test would fail if they disagreed.)

How the database does it: `EXPLAIN QUERY PLAN` (Day 12). `day19_sql.py` prints the plan for the indexed and the unindexed weather table:

```
EXPLAIN QUERY PLAN with wx:
   CORRELATED SCALAR SUBQUERY 1
   SEARCH w USING COVERING INDEX wx_ts (ts>? AND ts<?)

EXPLAIN QUERY PLAN with wx_plain:
   CORRELATED SCALAR SUBQUERY 1
   SCAN w
   USE TEMP B-TREE FOR ORDER BY
```

(Abridged.) With the index, each probe is a B-tree range seek; without it, a full scan of the weather table (plus a temporary sort for the `ORDER BY`) per AHU row. SQLite also builds a throwaway "automatic index" for the final `LEFT JOIN` on the unindexed copy, but not for the subquery's range condition, which is the expensive part.

Plain Python, two ways. In the pandas script. The nested loop is what SQLite does with no index. The `bisect` version is the same idea as the index: keep the weather timestamps sorted and binary-search them. The pandas script prints `bisect loop finds the same weather row as merge_asof: True`.

Timing, growing the data. The two O(N·M) methods stop at 10,000 rows; at a million rows they would run for hours. `NaN` means "not run". On the machine this was written on (seconds):

```
      N     M  nested_s  bisect_s    asof_s          (day19_pandas.py)
   1000    70  0.002315 0.0001861 0.0009288
  10000   726      0.25   0.00239  0.001678
 100000  7322       NaN   0.03476  0.008587
1000000 73532       NaN    0.4317   0.06423

      N     M  sql_noindex_s  sql_index_s             (day19_sql.py)
   1000    70        0.00362    0.0002054
  10000   726         0.3633     0.002943
 100000  7322            NaN      0.02833
1000000 73532            NaN       0.3345
```

Read it in three parts. **The unindexed methods** (`nested_s`, `sql_noindex_s`) grow by about 100x for every 10x of data (N grows 10x and so does M, so N·M grows 100x): 10,000 rows takes 0.25 s and 0.36 s, and a million would take hours. SQLite's unindexed subquery is a compiled nested loop, but it follows the same curve as the Python one (and here is not even faster). **With an index** (`sql_index_s`, and `bisect_s`, the Python equivalent) the growth is about 10x per 10x: N lookups, each O(log M). **`merge_asof`** is the quickest because it does the whole job as one O(N + M) merge in compiled code. It has almost no per-row overhead. Both scripts print the number of matched rows at every size (`timing_matches`), and the unit test checks that they are the same.

The lesson that carries over to all of Week 4: an index (or a sort) changes the *shape* of the curve. Vectorizing only changes the slope.

A sensor-versus-weather rule. In the spirit of `OAT-METEO`: while the fan runs, flag the AHU when its outdoor sensor differs from the weather by more than 3 °C, confirmed for 6 rows (30 minutes). No weather (stale or missing) means *unknown*, so no fault. In SQL, the join result is a CTE under the streak machinery from Day 14; in pandas, the same `shift` / `cumsum` / `cumcount` trick.

```
equipment_id  fault_rows  raw_rows  fault_hours
       AHU_1          27       111         2.25
       AHU_2         196       526        16.33
```

AHU_2's sensor is found: 16.3 confirmed hours against 2.3 for AHU_1, whose sensor the weather was derived from (it isn't silent, because the weather has 0.5 °C of noise and the rule's tolerance is finite). In real data there would be more false alarms, which is why the rule has a 3 °C tolerance, a confirmation time and a stale-weather guard. Note what the 2-hour tolerance bought: when the weather feed is down, the rule says nothing, instead of comparing a live sensor to yesterday's weather.

Try it

1. Tighten the tolerance to 1 hour. How many AHU rows lose their weather?
2. Replace "latest at or before" with "nearest *after*": SQL `w.ts >= h.ts ... ORDER BY w.ts ASC`, pandas `direction="forward"`. When would you ever want that, and when is it a bug?

Answers

1. `rows_without_weather` in each script's `RESULTS` already answers it: 1,250 rows with a 2-hour tolerance, 8,252 with one hour. Rows lose weather in the hours after a feed outage: with a 1-hour limit, a row whose latest reading is 1-2 hours old becomes unknown. The two implementations agree row for row. In SQL, `tol` is a parameter: `query(NO_WEATHER_SQL, tol=3600)`; in pandas, `join_pandas(ahu, wx, 3600)`.

2. In pandas, `pd.merge_asof(left, wx.assign(wx_ts=wx.ts), on="ts", direction="forward", tolerance=TOL_S)`. In SQL, swap the condition and the sort in the subquery:

   ```
   WHERE w.ts >= h.ts AND w.ts <= h.ts + :tol
   ORDER BY w.ts ASC LIMIT 1
   ```

   Every matched weather timestamp is then at or after the row's (1,272 rows end up with no match, the ones before a gap this time). The forward join pairs each row with a *future* weather reading. That is

legitimate for offline analysis when a value is stamped at the *end* of its interval (a mean over the previous hour). It is a **look-ahead bug** in anything that claims to run live, or in a backtest of a live rule: the rule would "know" something it couldn't have known yet. Always ask which direction is honest for the data you're joining.

Why it matters

"Latest value at or before" is the most common join in building data: setpoints that change a few times a day against readings every minute, schedules against measurements, weather against everything. Remember the three shapes: a nested loop (O(N·M)), an index lookup (O(N log M)), a sorted merge (O(N + M)). Day 20 pits the three tools against each other on the FC1 rule at scale.

day19_pandas.py

```
"""Day 19, pandas side: the as-of join (latest weather at or before
        each row) and a weather rule.

    uv run python lessons/unit2-python/day19_pandas.py

Each 5-minute AHU row gets the most recent weather reading at or before
        its time, but only if that
reading is at most TOL_S old (otherwise: unknown). pandas does it with
        `merge_asof`; plain Python
with a sorted list and binary search (`bisect`) or a nested loop. Then
        an OAT-METEO style rule: flag
the AHU while its fan runs and its outdoor sensor differs from the
        weather by more than DIFF_C.

AHU rows come from the Building 50 CSVs. The weather is a stand-in feed
        made from AHU_1's own
outdoor sensor (see make_weather). The timing section uses synthetic
        rows so the size can grow.
Compare with day19_sql.py.
"""

import sys
from bisect import bisect_right
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
DATA = ROOT / "data" / "BUILDING_50"
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

TOL_S = 2 * 3600  # weather older than this is stale
DIFF_C = 3.0  # sensor-vs-weather disagreement, degrees C
CONFIRM_ROWS = 6  # 6 x 5 min = 30 minutes
STEP_H = 5 / 60  # one row = 5 minutes, in hours


def make_weather(ts, temp_c, seed=3):
    """A stand-in weather feed: hourly means of a sensor (+ 0.5 C
        noise), with some hours missing.

    Real weather never matches a sensor exactly, and real feeds have
        failures: 12% of the hours are
    deleted and one 6-hour block is missing. In production this table
        is the Open-Meteo feed.
    """
    rng = np.random.default_rng(seed)
    hour = (np.asarray(ts) // 3600) * 3600
    wx = pd.DataFrame({"ts": hour, "t":
        np.asarray(temp_c)}).groupby("ts").t.mean()
    wx = wx.rename("wx_oat").reset_index()
    wx["wx_oat"] = (wx.wx_oat + rng.normal(0, 0.5, len(wx))).round(2)
    wx = wx[rng.random(len(wx)) >= 0.12]
    return wx[~wx.index.isin(range(len(wx) // 2, len(wx) // 2 +
        6))].reset_index(drop=True)
```

```
# 1. Load: both AHUs' CSVs, only the columns we need. Weeks 1-4 hard-
        code the column names.
COLUMNS = ["timestamp_utc", "supply_fan_status", "outside_air_temp_f"]
frames = []
for equip in ("AHU_1", "AHU_2"):
    f = pd.read_csv(DATA / equip / "history_wide.csv", usecols=COLUMNS)
    f.insert(0, "equipment_id", equip)
    frames.append(f)
ahu = pd.concat(frames, ignore_index=True)
ahu["ts"] = pd.to_datetime(ahu["timestamp_utc"],
        utc=True).dt.as_unit("s").astype("int64")
ahu["oat_c"] = (
    (ahu["outside_air_temp_f"] - 32.0) * 5.0 / 9.0
)  # the CSV is in degF; fault math is SI
ahu = ahu.rename(columns={"supply_fan_status": "fan_status"})
ahu = ahu[["equipment_id", "ts", "oat_c", "fan_status"]]

# 2. The stand-in weather feed, from AHU_1's own sensor
a1 = ahu[ahu.equipment_id == "AHU_1"]
wx = make_weather(a1.ts, a1.oat_c)
print(f"{len(ahu):,} AHU rows, {len(wx):,} weather rows (of {len(a1) *
        300 // 3600:,} hours)")


# 3. The as-of join. merge_asof needs both sides sorted by the key. Our
        table has two AHUs
#    interleaved by time, so sort by ts for the merge. wx_ts shows
        which weather row matched.
def join_pandas(ahu, wx, tol_s=TOL_S):
    left = ahu.sort_values("ts", kind="mergesort")
    right = wx.assign(wx_ts=wx.ts).sort_values("ts")
    out = pd.merge_asof(left, right, on="ts", direction="backward",
        tolerance=tol_s)
    return out[["equipment_id", "ts", "oat_c", "fan_status", "wx_ts",
        "wx_oat"]]


asof_join = join_pandas(ahu, wx)
print(asof_join.sort_values(["equipment_id",
        "ts"]).head(3).to_string(index=False))
print("rows with no usable weather:",
        int(asof_join.wx_ts.isna().sum()))


# 4. Sensor-versus-weather rule. No weather (stale or missing) means
        unknown, so no fault. The
#    confirmation is Day 14's streak machinery: streak ids by cumsum,
        rows inside by cumcount.
def rule_pandas(joined, diff_c=DIFF_C, confirm=CONFIRM_ROWS):
    j = joined.sort_values(["equipment_id",
        "ts"]).reset_index(drop=True)
    eq = j.equipment_id
    raw = ((j.fan_status == 1) & j.wx_oat.notna() & ((j.oat_c -
        j.wx_oat).abs() > diff_c)).astype(
        int
    )
```

```
    streak =
        raw.ne(raw.groupby(eq).shift()).astype(int).groupby(eq).cumsum()
    length = raw.groupby([eq, streak]).cumcount() + 1
    fault = ((raw == 1) & (length >= confirm)).astype(int)
    return j[["equipment_id", "ts", "wx_oat"]].assign(raw_fault=raw,
        streak_id=streak, fault=fault)


rule = rule_pandas(asof_join)
rule_hours = (
    rule.groupby("equipment_id")
    .agg(fault_rows=("fault", "sum"), raw_rows=("raw_fault", "sum"))
    .assign(fault_hours=lambda d: d.fault_rows * STEP_H)
    .reset_index()
)

# 5. Try it 1: tighten the tolerance to 1 hour. How many rows lose
        their weather?
no_weather = pd.DataFrame(
    [
        {"tol_s": t, "rows_without_weather": int(join_pandas(ahu, wx,
         t).wx_ts.isna().sum())}
        for t in (TOL_S, 3600)
    ]
)


# 6. Two ways in plain Python. The nested loop is what a database does
        with no index; the bisect
#    version is the same idea as an index: keep the weather timestamps
        sorted and binary-search.
def nested_loop(ts_list, wx_ts_list, tol_s=TOL_S):
    out = []
    for ts in ts_list:
        best = None
        for w in wx_ts_list:  # scans every weather row: O(M) per AHU
         row
            if w <= ts and (best is None or w > best):
                best = w
        out.append(best if best is not None and ts - best <= tol_s else
         None)
    return out


def bisect_loop(ts_list, wx_ts_list, tol_s=TOL_S):
    out = []
    for ts in ts_list:
        i = bisect_right(wx_ts_list, ts) - 1  # O(log M); wx_ts_list
         must be sorted
        out.append(wx_ts_list[i] if i >= 0 and ts - wx_ts_list[i] <=
         tol_s else None)
    return out


ts_list, wx_ts_list = ahu.ts.tolist(), sorted(wx.ts.tolist())
bisect_ts = pd.Series(bisect_loop(ts_list, wx_ts_list),
        dtype="float64")
key = pd.MultiIndex.from_arrays([ahu.equipment_id, ahu.ts])
want = asof_join.set_index(["equipment_id", "ts"]).wx_ts.reindex(key)
```

```
bisect_agrees = np.array_equal(bisect_ts.to_numpy(), want.to_numpy(),
        equal_nan=True)

# 7. Timing on synthetic rows (one AHU; weather one hour per 12 rows).
        The two O(N*M) methods stop
#    at 10,000 rows; at a million rows they would run for hours. NaN
        means "not run".
timing, matches = [], []
for n in lab.sizes([1_000, 10_000, 100_000, 1_000_000]):
    d = lab.synthetic_history(n)
    w = make_weather(d.ts, d.oat)
    reps = 3 if n <= 100_000 else 1
    tl, wl = d.ts.tolist(), sorted(w.ts.tolist())
    left = d[["ts"]]
    right = w.assign(wx_ts=w.ts)

    def asof(left=left, right=right):
        return pd.merge_asof(left, right, on="ts",
         direction="backward", tolerance=TOL_S)

    row = {"N": n, "M": len(w)}
    if n <= 10_000:
        row["nested_s"] = lab.best_time(lambda tl=tl, wl=wl:
         nested_loop(tl, wl), reps)
    row["bisect_s"] = lab.best_time(lambda tl=tl, wl=wl:
         bisect_loop(tl, wl), reps)
    row["asof_s"] = lab.best_time(asof, reps)
    timing.append(row)
    matches.append({"N": n, "M": len(w), "matched":
         int(asof().wx_oat.notna().sum())})
timing = pd.DataFrame(timing).reindex(columns=["N", "M", "nested_s",
        "bisect_s", "asof_s"])
timing_matches = pd.DataFrame(matches)

RESULTS = {
    "asof_join": asof_join,
    "oat_meteo_hours": rule_hours,
    "rows_without_weather": no_weather,
    "timing_matches": timing_matches,
}

print("\nSensor vs weather rule (fan running, |sensor - weather| > 3 C
        for 30 minutes):")
print(rule_hours.round(2).to_string(index=False))
print("\nRows without weather by tolerance:")
print(no_weather.to_string(index=False))
print("\nTiming, seconds (nested loop and bisect are plain Python;
        times vary by machine):")
print(timing.to_string(index=False, float_format=lambda x: f"
        {x:,.4g}"))
print("\nMatched rows at every size (the SQL script finds the same
        numbers):")
print(timing_matches.to_string(index=False))
print("\nchecks (printed, not asserted):")
print("  bisect loop finds the same weather row as merge_asof:",
        bisect_agrees)
```

day19_sql.py

```
"""Day 19, SQL side: the as-of join (latest weather at or before each
        row) and a weather rule.

    uv run python lessons/course.py setup         # once: loads the
        CSVs into var/lessons.sqlite
    uv run python lessons/unit2-python/day19_sql.py

The same computation as day19_pandas.py on the same rows. For each 5-
        minute AHU row, a correlated
subquery looks up the newest weather row at or before its time, but
        only one at most TOL_S old. With
an index on the weather timestamps each lookup is a B-tree range seek:
        O(N log M). Without the index
it scans the whole weather table per row: O(N*M). `EXPLAIN QUERY PLAN`
        shows the difference.

The weather is a stand-in feed made from AHU_1's own outdoor sensor
        (see make_weather). pandas
builds it as input data and it is loaded into the `wx` table. SQLite
        does the joins and the rule.
"""

import os
import sqlite3
import sys
from pathlib import Path

import numpy as np
import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

DB = Path(os.environ.get("LESSONS_DB", ROOT / "var" /
        "lessons.sqlite"))
if not DB.exists():
    raise SystemExit(f"{DB} not found. Run:  uv run python
        lessons/course.py setup")
conn = sqlite3.connect(DB)

PARAMS = {
    "tol": 2 * 3600,  # weather older than this is stale
    "diff": 3.0,  # sensor-vs-weather disagreement, degrees C
    "confirm": 6,  # 6 x 5 min = 30 minutes
}


def make_weather(ts, temp_c, seed=3):
    """A stand-in weather feed: hourly means of a sensor (+ 0.5 C
        noise), with some hours missing.

    Real weather never matches a sensor exactly, and real feeds have
        failures: 12% of the hours are
    deleted and one 6-hour block is missing. In production this table
        is the Open-Meteo feed.
    """
```

```
    rng = np.random.default_rng(seed)
    hour = (np.asarray(ts) // 3600) * 3600
    wx = pd.DataFrame({"ts": hour, "t":
         np.asarray(temp_c)}).groupby("ts").t.mean()
    wx = wx.rename("wx_oat").reset_index()
    wx["wx_oat"] = (wx.wx_oat + rng.normal(0, 0.5, len(wx))).round(2)
    wx = wx[rng.random(len(wx)) >= 0.12]
    return wx[~wx.index.isin(range(len(wx) // 2, len(wx) // 2 +
         6))].reset_index(drop=True)


# 1. Load: both AHUs' tables (the CSVs), the columns we need, in SI. ts
         is epoch seconds
#    (unixepoch parses the text timestamps). Setup: a temp table with
         an index.
conn.execute("DROP TABLE IF EXISTS temp.h")
conn.execute(
    """
CREATE TEMP TABLE h AS
SELECT 'AHU_1' AS equipment_id, unixepoch(timestamp_utc) AS ts,
       (outside_air_temp_f - 32.0) * 5.0 / 9.0 AS oat_c,
         supply_fan_status AS fan_status
FROM ahu_1
UNION ALL
SELECT 'AHU_2', unixepoch(timestamp_utc), (outside_air_temp_f - 32.0) *
         5.0 / 9.0,
       supply_fan_status
FROM ahu_2
"""
)
conn.execute("CREATE INDEX temp.h_eq_ts ON h (equipment_id, ts)")

# 2. The stand-in weather feed, from AHU_1's own sensor. Two copies of
         the same rows: `wx` with an
#    index on ts, `wx_plain` without.
a1 = pd.read_sql_query("SELECT ts, oat_c FROM h WHERE equipment_id =
         'AHU_1' ORDER BY ts", conn)
wx = make_weather(a1.ts, a1.oat_c)
for table in ("wx", "wx_plain"):
    conn.execute(f"DROP TABLE IF EXISTS temp.{table}")
    conn.execute(f"CREATE TEMP TABLE {table} (ts INTEGER, wx_oat
         REAL)")
    conn.executemany(f"INSERT INTO {table} VALUES (?, ?)",
         wx.itertuples(index=False, name=None))
conn.execute("CREATE INDEX temp.wx_ts ON wx (ts)")
n_rows = conn.execute("SELECT count(*) FROM h").fetchone()[0]
print(f"{n_rows:,} AHU rows, {len(wx):,} weather rows (of {len(a1) *
         300 // 3600:,} hours)")


# 3. The as-of join. The inner query runs once per AHU row, which makes
         it "correlated".
def join_sql(table="wx"):
    return f"""
SELECT j.equipment_id, j.ts, j.oat_c, j.fan_status, j.wx_ts, w.wx_oat
FROM (
  SELECT h.equipment_id, h.ts, h.oat_c, h.fan_status,
         (SELECT w.ts FROM {table} w
          WHERE w.ts <= h.ts AND w.ts >= h.ts - :tol
```

```
          ORDER BY w.ts DESC LIMIT 1) AS wx_ts
  FROM h
) j
LEFT JOIN {table} w ON w.ts = j.wx_ts
"""


# 4. Sensor-versus-weather rule: the join as a CTE under Day 14's
        streak machinery.
#    No weather (stale or missing) means unknown, so no fault.
RULE_SQL = f"""
WITH j AS ({join_sql()}), base AS (
  SELECT equipment_id, ts, wx_oat,
         CASE WHEN fan_status = 1 AND wx_oat IS NOT NULL AND ABS(oat_c
         - wx_oat) > :diff
              THEN 1 ELSE 0 END AS raw_fault
  FROM j
), marks AS (
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts)
                 THEN 0 ELSE 1 END AS new_streak
  FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
         ts) AS streak_id FROM marks
), ranked AS (
  SELECT *, ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id
         ORDER BY ts) AS streak_len
  FROM streaks
)
SELECT equipment_id, ts, wx_oat, raw_fault, streak_id,
       CASE WHEN raw_fault = 1 AND streak_len >= :confirm THEN 1 ELSE 0
         END AS fault
FROM ranked
"""
HOURS_SQL = f"""
SELECT equipment_id, SUM(fault) AS fault_rows, SUM(raw_fault) AS
         raw_rows,
       SUM(fault) * 5 / 60.0 AS fault_hours
FROM ({RULE_SQL})
GROUP BY equipment_id ORDER BY equipment_id
"""


def query(sql, **override):
    return pd.read_sql_query(sql, conn, params={**PARAMS, **override})


asof_join = query(join_sql())
print(asof_join.sort_values(["equipment_id",
         "ts"]).head(3).to_string(index=False))
print("rows with no usable weather:",
         int(asof_join.wx_ts.isna().sum()))
rule_hours = query(HOURS_SQL).astype({"fault_rows": "int64",
         "raw_rows": "int64"})
rule_hours = rule_hours[["equipment_id", "fault_rows", "raw_rows",
         "fault_hours"]]
```

```
# 5. Try it 1: tighten the tolerance to 1 hour. How many rows lose
        their weather?
NO_WEATHER_SQL = f"SELECT count(*) - count(wx_ts) AS n FROM
        ({join_sql()})"
no_weather = pd.DataFrame(
    [
        {"tol_s": t, "rows_without_weather": int(query(NO_WEATHER_SQL,
         tol=t).n[0])}
        for t in (7200, 3600)
    ]
)

# 6. How the database does it: EXPLAIN QUERY PLAN (Day 12). With the
        index, each probe is a B-tree
#    range seek; without it, a full scan of the weather table per AHU
        row.
plans = {t: query("EXPLAIN QUERY PLAN " + join_sql(t)).detail.tolist()
         for t in ("wx", "wx_plain")}

# 7. Timing on synthetic rows (one AHU; weather one hour per 12 rows).
        The unindexed query is
#    O(N*M), so it stops at 10,000 rows. NaN means "not run". Loading
        the tables is setup.
COUNT_SQL = """
SELECT COUNT(wx_ts) AS matched FROM (
  SELECT (SELECT w.ts FROM {t} w WHERE w.ts <= h.ts AND w.ts >= h.ts -
        :tol
          ORDER BY w.ts DESC LIMIT 1) AS wx_ts
  FROM h)
"""
timing, matches = [], []
for n in lab.sizes([1_000, 10_000, 100_000, 1_000_000]):
    d = lab.synthetic_history(n)
    w = make_weather(d.ts, d.oat)
    mem = sqlite3.connect(":memory:")
    lab.to_sqlite(d[["ts"]], mem, "h")  # the timing only needs the
        timestamps
    lab.to_sqlite(w, mem, "wx", index=["ts"])  # indexed
    lab.to_sqlite(w, mem, "wx_plain")  # the same rows, no index
    reps = 3 if n <= 100_000 else 1
    row = {"N": n, "M": len(w)}

    def run(table, mem=mem):
        return mem.execute(COUNT_SQL.format(t=table), {"tol":
         PARAMS["tol"]}).fetchone()[0]

    row["sql_index_s"] = lab.best_time(lambda run=run: run("wx"), reps)
    if n <= 10_000:  # the O(N*M) case
        row["sql_noindex_s"] = lab.best_time(lambda run=run:
         run("wx_plain"), reps)
    timing.append(row)
    matches.append({"N": n, "M": len(w), "matched": run("wx")})
    mem.close()
timing = pd.DataFrame(timing).reindex(columns=["N", "M",
        "sql_noindex_s", "sql_index_s"])
timing_matches = pd.DataFrame(matches)

RESULTS = {
    "asof_join": asof_join,
```

```
    "oat_meteo_hours": rule_hours,
    "rows_without_weather": no_weather,
    "timing_matches": timing_matches,
}

print("\nSensor vs weather rule (fan running, |sensor - weather| > 3 C
        for 30 minutes):")
print(rule_hours.round(2).to_string(index=False))
print("\nRows without weather by tolerance:")
print(no_weather.to_string(index=False))
for table, plan in plans.items():
    print(f"\nEXPLAIN QUERY PLAN with {table}:")
    for line in plan:
        print("  ", line)
print("\nTiming, seconds (times vary by machine):")
print(timing.to_string(index=False, float_format=lambda x: f"
        {x:,.4g}"))
print("\nMatched rows at every size (the pandas script finds the same
        numbers):")
print(timing_matches.to_string(index=False))
```

Day 20 — Scaling showdown, and the Week 4 challenge

Goal: run Day 14's FC1 confirmation-streak rule as a plain Python state machine, as vectorized pandas and as SQLite window SQL on data that grows 10x per step; compare the times and how they grow; and decide, with numbers, when each one is the right tool.

Read

This is the same rule as Day 14: raw fault = fan running and duct static more than 25 Pa below setpoint; a fault is **confirmed** once the raw fault has held for 2 consecutive rows. All the rule's cleverness is in "consecutive": each row's answer depends on the rows before it. That's the part that makes the tools differ.

	Plain Python (`day20_pandas.py`)	**pandas** (`day20_pandas.py`)	**SQLite** (`day20_sql.py`)
Idea	one pass, a tiny state machine: `streak_len`	column ops: `shift`, `cumsum`, `cumcount`	window functions: `LAG`, `SUM OVER`, `ROW_NUMBER`
State	one integer, carried along	none: streak ids are *recomputed* with `cumsum`	none: window frames recompute it
Work	1 pass, interpreter speed	~5 passes over whole arrays, C speed	sort/partition, several window passes
Memory	O(1) (streams the rows)	O(n) (needs all rows in arrays)	the database engine's (not in Python)
Data starts	in Python objects	has to be loaded first	where it already is

A state machine is the natural way to say "count consecutive rows" in a loop, and the awkward way to say it in column operations (Day 13 needed a trick: compare with the previous row, running-sum the changes). SQL and pandas are both doing that trick. Whether the trick costs more than the interpreter overhead is what the tables below show.

The data is `lab.synthetic_history`, the same rows in both scripts. The loop in `day20_pandas.py` resets its state when the AHU changes (like `PARTITION BY` does for SQL), so it can be checked row by row on two AHUs.

Run

```
uv run python lessons/course.py run 20
```

That runs `day20_pandas.py` (the loop and the pandas rule), then `day20_sql.py` (the window SQL). It is the longest pair of the week (about 35 seconds in all: the million-row steps dominate). `RESULTS` in both holds `per_row` (every row's confirmed-fault flag on two AHUs of 5,000 rows), `totals_by_size` (confirmed rows and episodes at every benchmark size), `first_fault` (the first confirmed fault in a table) and `longest_episode` (the challenge below). Compared with each other by the unit test; you compare the timings.

Both scripts print the totals at every size, the same in both:

```
   rows  fault_rows  episodes
   1000          72         1
  10000         479         4
 100000        4924        36
1000000       49684       348
```

`streak_id` restarts at 1 for each AHU (the window is partitioned), so counting episodes across AHUs needs `COUNT(DISTINCT equipment_id || ':' || streak_id)` in SQL; counting plain `streak_id` merges different AHUs' episodes into one. In the loop, the equivalent is the `if eq != cur_eq:` reset at the top of the body: without it, a streak at the end of AHU_1 would carry into the start of AHU_2.

The showdown. Synthetic data, 10x more rows each step, one AHU. SQL loading is **setup** and timed apart from the query (`sql_load`). On the machine this was written on (seconds):

```
              loop    pandas              (day20_pandas.py)
rows
1000     0.0005124  0.002609
10000     0.005097  0.003933
100000     0.05864   0.03202
1000000     0.5769    0.2743

         sql_load sql_window              (day20_sql.py)
rows
1000     0.005301   0.005588
10000     0.02772    0.04336
100000     0.2854     0.5476
1000000     3.042      5.953
```

and the growth table each script prints (time at this size over time at the previous, 10x smaller size). How to read it: about 10 means O(n): ten times the data, ten times the time. Smaller than 10 at the first rows means fixed overhead (starting a query, allocating arrays) is still a big part of the time. Bigger than 10 would mean worse than linear: sorting adds a log factor, and

once arrays stop fitting in CPU cache each pass slows down. Here, the loop's growth is 10-12 at every step (1 pass at interpreter speed), pandas is 1.5, then 8, then 9 as it leaves the overhead regime, and the window SQL is 8 to 13 (that last bit of extra is the sort).

At a million rows: pandas on resident data 0.27 s, the plain loop 0.58 s, window SQL 6.0 s (plus 3.0 s to load it, a cost Day 16 measured). Three readings of the same table:

- *If the data is already a DataFrame*, pandas wins, by about 2x over the loop. The loop's penalty is smaller than on Day 16's trivial counting because a state machine is *one* pass, while the pandas version needs several full-array passes (`shift`, `ne`, `cumsum`, `groupby.cumcount`). The vectorized version is still quicker, but not by 50x.
- *If the data lives in the database*, add Day 16's cost of moving it out (2.5 s for a million rows) to the pandas number before comparing; the loop pays a similar price to pull rows out of a cursor.
- *Window SQL is the slowest of all here*, about 20x pandas and 10x the loop: SQLite sorts the rows, then makes four window passes, each of which materializes intermediate rows. SQL's strength is not this kind of row-to-row logic. It is the *selective* query that an index answers without reading most of the table (Day 12), and the aggregate that runs where the data is (Day 16).

So when does each one win?

- **SQL** when the data already lives in the database and you want a small answer: a total, a top-10, one AHU's last week. The index finds the rows without reading the rest (Day 12), nothing is copied into Python, and the result is one small table. Also when many people or programs need the same logic: the query ships with the data. It loses on cost-per-row for complicated row-to-row logic, and it is awkward to step through.
- **pandas** for in-memory column math and exploratory work: it is the fastest once the data is in arrays, and the interactive loop (change a line, look at a plot) is why it's the oracle in Open-FDD's parity tests. It loses when the data doesn't fit in memory (Day 17), or when moving the data in costs more than the calculation (Day 16).
- **A plain loop** when the data streams past once and the state is complicated or the data is small: it needs O(1) memory (Day 17), it can **stop early** (next), and a state machine with ten fields is easier to read than ten window functions. Its per-row cost is high, but it is flat. If a loop is the right *shape* and merely slow, tools like numba or Cython compile exactly this kind of loop to machine code; that's a story for another course.

Open-FDD ships the pandas version as the reference and the SQL version where the data lives. Both are correct, and the parity test is what lets you pick per situation.

Try it

Stop early. How long does each version take to find the *first* confirmed fault timestamp in a 200,000-row table (20,000 under the test)? Compare the loop that returns at the first hit with the loop that reads everything, with pandas, and with the window SQL. The scripts print it, and both find the same timestamp (`first_fault`).

Answer

On the machine this was written on, for 200,000 rows:

```
loop, early exit              0.0389 s
loop, whole table             0.1198 s
pandas (computes every row)   0.0745 s
SQL, MIN(ts) over the faults  1.0152 s       (the totals query: 1.0804
s)
```

The early-exit loop does a fraction of the work, because the first confirmed fault is about a third of the way into the table (that is where the synthetic weak-fan window starts), so two thirds are never read. It is faster than the vectorized pandas version here, though pandas beats the full loop. pandas and the window SQL have no equivalent: they compute every row's state before you can ask for the first. `SELECT MIN(ts)` takes as long as the whole query (and a `LIMIT 1` would not help for the same reason). Early exit is the loop's unique strength: it would matter far more if the first fault were near the start or each row cost a network call. The loop's win is in *not needing the rest of the data at all*.

Week 4 challenge

Find the **longest confirmed fault episode** for each AHU: its length in rows and in minutes, on a table with two AHUs of 20,000 rows each (5,000 under the test). An episode is a maximal run of consecutive raw-fault rows that lasts at least `CONFIRM` rows. Implement it three ways, and say which one you would ship if the data were (a) one building's database, (b) a notebook of a month of data, (c) a stream on a controller.

- **Loop** (`longest_loop` in `day20_pandas.py`): a streaming loop that tracks the current and the best run, per AHU.
- **pandas** (`longest_pandas`): the size of each streak with `groupby([eq, streak])`, keep the raw-fault ones that last long enough, take the max per AHU.
- **SQL** (`LONGEST_SQL` in `day20_sql.py`): `GROUP BY equipment_id, streak_id HAVING COUNT(*) >= :confirm`, then `MAX`.

Both scripts print the same table (`longest_episode` in `RESULTS`), and the pandas script also checks its loop and pandas versions agree:

```
equipment_id  longest_rows  longest_min
       AHU_1           144        720.0
       AHU_2           144        720.0
```

Answer

Both AHUs have a longest episode of 144 rows = 12 hours: the 12-hour fan schedule (06:00-18:00) lies entirely inside the weak-fan window, so the whole run is one episode (synthetic data: a real building would be messier).

The key step in all three is the same trick as Day 14: a new streak starts where the raw flag changes, the running sum of starts numbers the streaks, and the size of a streak is its length. In pandas that is `raw.ne(raw.groupby(eq).shift())` then `cumsum`; in SQL `LAG` then `SUM() OVER`; in the loop, `run = run + 1 if raw == prev_raw else 1`.

Which to ship: (a) one building's database: **SQL**, next to the data, with an index, returning two numbers. (b) a notebook with a month of data: **pandas**, because you'll want to plot the episodes next. (c) a stream on a controller: the **loop**, with O(1) memory and early updates the moment each episode ends. All three give the same answer, which is the point of having two scripts to compare.

Week 4 in one sentence: the answer is the same everywhere, and *where the data is, how much of it you hold in memory, and what shape the logic has* decide which tool gets there cheapest.

day20_pandas.py

```
"""Day 20, pandas side: Day 14's FC1 confirmation streaks as a Python
        loop and as vectorized pandas.

    uv run python lessons/unit2-python/day20_pandas.py

The rule is Day 14's: raw fault = fan running and duct static more than
        25 Pa below setpoint; a
fault is confirmed once the raw fault has held for 2 consecutive rows.
        "Consecutive" is what makes
the tools differ: each row's answer depends on the rows before it.

Here: the plain Python state machine (one pass, O(1) state) and the
        pandas column trick (shift,
cumsum, cumcount), checked row by row, then timed on data that grows
        10x per step. Also the "stop
early" experiment and the Week 4 challenge (longest confirmed episode
        per AHU, three ways; the loop
and pandas versions are here, the SQL one in day20_sql.py). Same
        synthetic rows as day20_sql.py.
"""

import sys
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

TOL_PA = 25.0  # how far below setpoint counts as a fault
CONFIRM = 2  # rows (10 minutes at 5-minute spacing)
SIZES = lab.sizes([1_000, 10_000, 100_000, 1_000_000])
N_PARITY = 5_000  # rows per AHU for the row-by-row check
N_FIRST = 20_000 if lab.QUICK else 200_000  # rows for the stop-early
        experiment
N_CHALLENGE = 5_000 if lab.QUICK else 20_000  # rows per AHU for the
        challenge


# 1. The plain Python state machine: only streak_len survives from one
        row to the next.
def loop_rule(d):
    """d is sorted by (equipment_id, ts). Returns (per-row fault flags,
        fault_rows, episodes)."""
    flags = []
    cur_eq = prev_raw = None
    streak_len = fault_rows = episodes = 0
    for eq, fan, static, sp in zip(
        d.equipment_id, d.fan_status, d.duct_static, d.duct_static_sp,
         strict=True
    ):
        if eq != cur_eq:  # a new AHU: reset the state, as PARTITION BY
         does for SQL
            cur_eq, prev_raw, streak_len = eq, None, 0
        raw = 1 if fan == 1 and static < sp - TOL_PA else 0
        streak_len = streak_len + 1 if raw == prev_raw else 1
```

```
        prev_raw = raw
        fault = 1 if raw == 1 and streak_len >= CONFIRM else 0
        fault_rows += fault
        episodes += 1 if fault and streak_len == CONFIRM else 0  # the
         row where a streak confirms
        flags.append(fault)
    return flags, fault_rows, episodes


# 2. pandas: Day 14's column trick. A new streak starts where raw
        changes; cumsum numbers them.
def pandas_rule(d):
    """d is sorted by (equipment_id, ts). Returns (per-row fault
        Series, fault_rows, episodes)."""
    raw = ((d.fan_status == 1) & (d.duct_static < d.duct_static_sp -
        TOL_PA)).astype(int)
    eq = d.equipment_id
    streak =
        raw.ne(raw.groupby(eq).shift()).astype(int).groupby(eq).cumsum()
    length = raw.groupby([eq, streak]).cumcount() + 1
    fault = ((raw == 1) & (length >= CONFIRM)).astype(int)
    episodes = int(((raw == 1) & (length == CONFIRM)).sum())
    return fault, int(fault.sum()), episodes


# 3. Parity row by row, on two AHUs (so the reset / partition matters)
df2 = lab.synthetic_history(N_PARITY, equips=2)
flags_loop, loop_faults, loop_eps = loop_rule(df2)
fault_pd, pd_faults, pd_eps = pandas_rule(df2)
per_row = df2[["equipment_id", "ts"]].assign(fault=fault_pd)
print(
    f"two AHUs x {N_PARITY:,} rows: loop {loop_faults} / {loop_eps},
        pandas {pd_faults} / {pd_eps}"
)
print("loop and pandas flag the same rows:", flags_loop ==
        fault_pd.tolist())

# 4. The showdown: time both as the data grows 10x per step (one AHU)
timing, totals = [], []
for n in SIZES:
    d = lab.synthetic_history(n)
    reps = 3 if n <= 100_000 else 1
    _, faults, eps = pandas_rule(d)
    totals.append({"rows": n, "fault_rows": faults, "episodes": eps})
    timing.append(
        {
            "rows": n,
            "loop": lab.best_time(lambda d=d: loop_rule(d), reps),
            "pandas": lab.best_time(lambda d=d: pandas_rule(d), reps),
        }
    )
totals = pd.DataFrame(totals)
t = pd.DataFrame(timing).set_index("rows")
growth = (t / t.shift(1)).dropna()


# 5. Stop early: the loop can return at the first confirmed fault;
        pandas has to compute every row
```

```
#    before you can ask for the first one.
def first_fault_loop(d):
    streak_len, prev_raw = 0, None
    for fan, static, sp, ts in zip(
        d.fan_status, d.duct_static, d.duct_static_sp, d.ts,
         strict=True
    ):
        raw = 1 if fan == 1 and static < sp - TOL_PA else 0
        streak_len = streak_len + 1 if raw == prev_raw else 1
        prev_raw = raw
        if raw == 1 and streak_len >= CONFIRM:
            return ts  # early exit: the rest of the table is never
         read
    return None


def first_fault_pandas(d):
    fault, _, _ = pandas_rule(d)
    return int(d.ts[fault == 1].iloc[0])


dn = lab.synthetic_history(N_FIRST)
first_fault = pd.DataFrame({"first_fault_ts": [first_fault_loop(dn)]})
early = {
    "loop, early exit": lab.best_time(lambda: first_fault_loop(dn)),
    "loop, whole table": lab.best_time(lambda: loop_rule(dn)),
    "pandas (computes every row)": lab.best_time(lambda:
         first_fault_pandas(dn)),
}


# 6. Week 4 challenge, loop and pandas: the longest confirmed episode
         per AHU, in rows and minutes.
#    An episode is a maximal run of raw-fault rows that lasts at least
         CONFIRM rows.
def longest_loop(d):
    best = {}
    cur_eq = prev_raw = None
    run = 0
    for eq, fan, static, sp in zip(
        d.equipment_id, d.fan_status, d.duct_static, d.duct_static_sp,
         strict=True
    ):
        raw = 1 if fan == 1 and static < sp - TOL_PA else 0
        if eq != cur_eq:
            cur_eq, prev_raw, run = eq, None, 0
            best.setdefault(eq, 0)
        run = run + 1 if raw == prev_raw else 1
        prev_raw = raw
        if raw == 1 and run >= CONFIRM:
            best[eq] = max(best[eq], run)
    return pd.DataFrame({"equipment_id": list(best), "longest_rows":
         list(best.values())})


def longest_pandas(d):
    raw = ((d.fan_status == 1) & (d.duct_static < d.duct_static_sp -
         TOL_PA)).astype(int)
```

```
    eq = d.equipment_id
    streak =
        raw.ne(raw.groupby(eq).shift()).astype(int).groupby(eq).cumsum()
    runs = raw.groupby([eq, streak]).agg(["first", "size"])  # one row
        per streak
    episodes = runs[(runs["first"] == 1) & (runs["size"] >= CONFIRM)]
        ["size"]
    return
        episodes.groupby("equipment_id").max().rename("longest_rows").reset_ind


dc = lab.synthetic_history(N_CHALLENGE, equips=2)
longest_l, longest_p = longest_loop(dc), longest_pandas(dc)
longest = longest_p.assign(longest_min=longest_p.longest_rows * 5.0)

RESULTS = {
    "per_row": per_row,
    "totals_by_size": totals,
    "first_fault": first_fault,
    "longest_episode": longest,
}

print("\nFC1 totals at every size (the SQL script finds the same
        numbers):")
print(totals.to_string(index=False))
print("\nseconds (times vary by machine; read the shape, not the
        digits)")
print(t.map(lambda x: f"{x:,.4g}").to_string())
print("\ngrowth: time at this size / time at the previous (10x smaller)
        size")
print(growth.round(1).to_string())
print(f"\nFirst confirmed fault in {N_FIRST:,} rows: ts
        {first_fault.first_fault_ts[0]}")
for name, secs in early.items():
    print(f"  {name:<30} {secs:.4f} s")
print(f"\nLongest confirmed episode per AHU ({N_CHALLENGE:,} rows
        each):")
print(longest.to_string(index=False))
print("\nchecks (printed, not asserted):")
print("  longest episode, loop == pandas:",
        longest_l.equals(longest_p))
print("  first fault, loop == pandas:", first_fault_loop(dn) ==
        first_fault_pandas(dn))
big = t.iloc[-1]
print(f"  at {t.index[-1]:,} rows: loop {big['loop']:.3f} s, pandas
        {big['pandas']:.3f} s")
print("  loop is slower than pandas at the largest size:",
        bool(big["loop"] > big["pandas"]))
```

day20_sql.py

```
"""Day 20, SQL side: Day 14's FC1 confirmation streaks as SQLite window
        functions, at scale.

    uv run python lessons/unit2-python/day20_sql.py

The rule is Day 14's: raw fault = fan running and duct static more than
        25 Pa below setpoint; a
fault is confirmed once the raw fault has held for 2 consecutive rows.
        SQLite does it with window
functions: LAG marks where the value changes, a running SUM numbers the
        streaks, ROW_NUMBER counts
the rows inside each streak. It sorts and partitions, then makes
        several window passes.

Timed on data that grows 10x per step (loading is setup, timed apart
        from the query), plus the "stop
early" experiment (SQL has no early exit: the window query computes
        every row first) and the Week 4
challenge (Longest confirmed episode per AHU, as GROUP BY the streak
        id). Same synthetic rows as
day20_pandas.py.
"""

import sqlite3
import sys
import time
from pathlib import Path

import pandas as pd

ROOT = Path(__file__).resolve().parents[2]
sys.path.insert(0, str(ROOT / "lessons"))
import lab  # noqa: E402

PARAMS = {"tol": 25.0, "confirm": 2}  # pascals; rows (10 minutes at 5-
        minute spacing)
SIZES = lab.sizes([1_000, 10_000, 100_000, 1_000_000])
N_PARITY = 5_000  # rows per AHU for the per-row result
N_FIRST = 20_000 if lab.QUICK else 200_000  # rows for the stop-early
        experiment
N_CHALLENGE = 5_000 if lab.QUICK else 20_000  # rows per AHU for the
        challenge

# One CTE chain; the queries below add a final SELECT to it.
RULE_SQL = """
WITH base AS (                                  -- gate + raw fault
  SELECT equipment_id, ts,
         CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
         :tol
              THEN 1 ELSE 0 END AS raw_fault
  FROM h
), marks AS (                                   -- a new streak starts
         where raw_fault changes
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
         equipment_id ORDER BY ts)
                 THEN 0 ELSE 1 END AS new_streak
  FROM base
```

```
), streaks AS (                                -- running sum of starts
        numbers the streaks
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
        ts) AS streak_id
  FROM marks
), ranked AS (                                 -- row number inside each
        streak
  SELECT *, ROW_NUMBER() OVER (PARTITION BY equipment_id, streak_id
        ORDER BY ts) AS streak_len
  FROM streaks
), per_row AS (
  SELECT equipment_id, ts, streak_id,
        CASE WHEN raw_fault = 1 AND streak_len >= :confirm THEN 1 ELSE
        0 END AS fault
  FROM ranked
)
"""
# streak_id restarts at 1 for each AHU, so episodes are counted per
        (AHU, streak)
PER_ROW_SQL = RULE_SQL + "SELECT equipment_id, ts, fault FROM per_row
        ORDER BY equipment_id, ts"
TOTALS_SQL = (
    RULE_SQL
    + """
SELECT COALESCE(SUM(fault), 0) AS fault_rows,
       COUNT(DISTINCT CASE WHEN fault = 1 THEN equipment_id || ':' ||
        streak_id END) AS episodes
FROM per_row
"""
)
FIRST_SQL = RULE_SQL + "SELECT MIN(ts) AS first_fault_ts FROM per_row
        WHERE fault = 1"

# Week 4 challenge: GROUP BY the streak id, keep the confirmed ones,
        take the MAX per AHU
LONGEST_SQL = """
WITH base AS (
  SELECT equipment_id, ts,
        CASE WHEN fan_status = 1 AND duct_static < duct_static_sp -
        :tol
             THEN 1 ELSE 0 END AS raw_fault
  FROM h
), marks AS (
  SELECT *, CASE WHEN raw_fault = LAG(raw_fault) OVER (PARTITION BY
        equipment_id ORDER BY ts)
                THEN 0 ELSE 1 END AS new_streak
  FROM base
), streaks AS (
  SELECT *, SUM(new_streak) OVER (PARTITION BY equipment_id ORDER BY
        ts) AS streak_id
  FROM marks
), episodes AS (
  SELECT equipment_id, streak_id, COUNT(*) AS rows_in_streak
  FROM streaks WHERE raw_fault = 1
  GROUP BY equipment_id, streak_id HAVING COUNT(*) >= :confirm
)
SELECT equipment_id, MAX(rows_in_streak) AS longest_rows,
        MAX(rows_in_streak) * 5.0 AS longest_min
FROM episodes GROUP BY equipment_id ORDER BY equipment_id
```

```
"""


def make(n, equips=1):
    """n rows per AHU of synthetic history in an in-memory table `h`
         with an index. Setup."""
    mem = sqlite3.connect(":memory:")
    lab.to_sqlite(lab.synthetic_history(n, equips=equips), mem, "h",
         index=["equipment_id", "ts"])
    return mem


def query(mem, sql):
    return pd.read_sql_query(sql, mem, params=PARAMS)


# 1. Per-row faults on two AHUs (so the PARTITION BY matters)
mem2 = make(N_PARITY, equips=2)
per_row = query(mem2, PER_ROW_SQL)
print(f"two AHUs x {N_PARITY:,} rows: {int(per_row.fault.sum())}
         confirmed fault rows")

# 2. The showdown: load once per size (timed apart), then time the
         window query (one AHU)
timing, totals = [], []
for n in SIZES:
    started = time.perf_counter()
    mem = make(n)
    t_load = time.perf_counter() - started
    reps = 3 if n <= 100_000 else 1
    totals.append({"rows": n, **query(mem,
         TOTALS_SQL).iloc[0].astype(int).to_dict()})
    timing.append(
        {
            "rows": n,
            "sql_load": t_load,
            "sql_window": lab.best_time(lambda mem=mem: query(mem,
         TOTALS_SQL), reps),
        }
    )
    mem.close()
totals = pd.DataFrame(totals)
t = pd.DataFrame(timing).set_index("rows")
growth = (t / t.shift(1)).dropna()

# 3. Stop early: SQL has no early exit. The window functions compute
         every row's state, so even a
#    MIN(ts) over the faults takes as long as the whole query (and
         LIMIT 1 would not change that).
memf = make(N_FIRST)
first_fault = query(memf, FIRST_SQL)
t_first = lab.best_time(lambda: query(memf, FIRST_SQL))
t_full = lab.best_time(lambda: query(memf, TOTALS_SQL))

# 4. Week 4 challenge: longest confirmed episode per AHU
memc = make(N_CHALLENGE, equips=2)
longest = query(memc, LONGEST_SQL)
```

```
RESULTS = {
    "per_row": per_row,
    "totals_by_size": totals,
    "first_fault": first_fault,
    "longest_episode": longest,
}

print("\nFC1 totals at every size (the pandas script finds the same
        numbers):")
print(totals.to_string(index=False))
print("\nseconds (times vary by machine; read the shape, not the
        digits)")
print(t.map(lambda x: f"{x:,.4g}").to_string())
print("\ngrowth: time at this size / time at the previous (10x smaller)
        size")
print(growth.round(1).to_string())
print(f"\nFirst confirmed fault in {N_FIRST:,} rows: ts
        {first_fault.first_fault_ts[0]}")
print(f"  SQL, MIN(ts) over the faults   {t_first:.4f} s")
print(f"  SQL, totals query              {t_full:.4f} s")
print(f"\nLongest confirmed episode per AHU ({N_CHALLENGE:,} rows
        each):")
print(longest.to_string(index=False))
big = t.iloc[-1]
print(f"\nobservation: at {t.index[-1]:,} rows the window query takes
        {big['sql_window']:.3f} s")
```

Reading — Week 4

The lessons first, then these for exact behavior.

- Python timeit
- Python tracemalloc
- pandas: scaling to large datasets
- pandas.merge_asof
- SQLite query optimizer overview
- Open-FDD SQL cookbook